

Blueprints with Teeth: Fail-Closed Architecture Conformance for AI-Built Systems

Design, Checker Validation, and First Production Experience of bce

Mitchell Tieleman*
Odin Labs

Abstract

The same codebase — roughly sixty Next.js route files on a live multi-tenant deployment, from which the extractor mints 75 handler components — graded twice by the same conformance engine, produced a score of 100 and *pass* under one architecture blueprint, and a score of 0 and *fail*, with 75 authentication-coverage violations, under another. Not a line of code differed; what differed was whether the blueprint could fail. We report on bce, a deterministic blueprint-conformance engine for AI-built systems, and its first production deployment. An operator-authored, PR-reviewed blueprint declares intended architecture; extractors build the observed architecture graph; policy evaluation grades the difference into a numeric score with a fail-closed verdict. Three mechanisms, to our knowledge not previously combined, keep that verdict honest: a **toothlessness gate** that refuses blueprints no realistic mutation can redden — vacuity detection applied at CI time; **checker self-validation**, measuring the engine’s own recall over a seeded-defect corpus spanning all eight enforcing constraint types — 25 defects at the 2026-08-02 freeze (recall 1.00, false-positive rate 0.00 over 35 reports), grown append-only to 34 defects including five Python-surface entries and re-measured at 1.00/0.00 over 45 reports on 2026-08-10, with a standing mutation hunt in which zero of eight seeded drift mutants stay green; and **served-runtime behavioral observations** graded inside a fail-closed required merge check, green- and red-proven in CI. Verdicts emit hash-chained, re-derivable evidence. To our knowledge this is the first production-deployed system closing this full loop. In its first two weeks as a required merge check across four repositories (advisory in a fifth) on the author’s own multi-tenant fleet, the engine executed 1,063 required-gate runs, and operation has continued at tempo since (16,438 blueprint-workflow runs by 2026-08-31); we report the operational record, five production incidents — including a structurally perfect blueprint caught certifying a behaviorally dead subsystem, remediated with input-conditioned behavioral invariants — and the lessons. The complete seeded-defect corpus, sample evidence records with a zero-dependency chain verifier, and the CI transcripts of the cited gate runs are publicly available at <https://github.com/odin-labs-ai/bce-paper-artifacts>; the engine’s open-core release is public at <https://github.com/blueprint-conformance/bce> (Apache-2.0; npm bce-engine).

1 Introduction

Autonomous coding agents now produce and modify production software at a tempo human review does not match — on our own fleet, the required conformance gates and their scheduled monitors alone executed roughly 85 times per day (§6). The consequence is not merely more bugs; it is

*AI-assisted-drafting disclosure: both the system and this manuscript were developed with substantial AI assistance, disclosed per venue policy; see the discussion in §7.

that the *architecture* of an AI-built system — which modules may depend on which, which surfaces may reach which providers, which handlers must enforce authentication — drifts silently away from its authors’ intent. Recent evidence suggests that supplying architectural documents to LLM-assisted generation substantially improves conformance of the generated code [8]; but an intent artifact that shapes generation upstream does nothing to *measure* the system that actually accrued downstream, commit after agent-authored commit. Governance of AI-generated code has become a central concern of 2026 tooling and research; what remains scarce is machinery that grades a built system against an authored intent artifact, fails closed, and leaves evidence that can be independently re-derived.

A motivating incident from our own fleet illustrates why measurement, not authorship, is the hard part. We graded the same codebase — roughly sixty Next.js `route.ts` files on a live multi-tenant deployment — twice, through the same engine, triggered from the product’s own verification surface (a button in the deployed product that runs the engine against the live tenant). Under the first blueprint: score 100, *pass*. Under the second, which added one enforceable authentication-coverage constraint (C-AUTH-02: every route handler must carry a satisfying guard edge): score 0, *fail*, 75 violations. The counts differ because the unit differs: the extractor mints one `apiRouteHandler` component per (file, HTTP-verb) pair, so a 60-file route surface yields 75 handler components, one violation each. Not one line of code differed between the runs. What differed was whether the blueprint had *teeth* — whether any state of the codebase could have made it fail. The first blueprint could not fail, so its perfect score certified nothing. A conformance verdict is only as honest as the machinery that could have produced a red one; that observation drives every design decision in this paper.

We report on **bce** (a *blueprint-conformance engine*) and its first production deployment; §3 and Figure 1 give the system. This paper makes four contributions:

1. **A production-deployed conformance loop (§3, §6).** To our knowledge — independently of the contemporaneous Spec Growth Engine design proposal [12] (§7) — the first production-deployed system closing the loop from authored-intent blueprint through deterministic scoring, fail-closed required-check gating, hash-chained re-derivable evidence, and served-runtime observation, with the checker itself validated by measurement.
2. **The toothlessness gate (§4.1).** A graph-mutation oracle that refuses blueprints unable to fail — the vacuity-detection lineage [5, 13] realized as CI-time admission control over architecture blueprints — with a full-portfolio audit: 33/33 fleet blueprints toothed; one genuine authored vacuity found at the constraint level (measured 2026-08-01/02 at engine 0.12.0; §4.1 reports the oracle-version caveat).
3. **Checker self-validation (§4.2–§4.3).** Measured recall of the checker itself over a seeded-defect corpus grown append-only from 25 defects (2026-08-02 freeze) to 34 — including five Python-surface defects behind a pluggable extraction-profile registry — covering all eight enforcing constraint types: recall 1.00 at false-positive rate 0.00 (35 reports at the freeze; re-measured 1.00/0.00 over 45 reports on 2026-08-10), a standing mutation hunt with zero surviving drift mutants, and a negative self-test demonstrating the measurement apparatus can itself go red.
4. **Production experience at agent tempo (§6).** 1,664 blueprint-workflow runs (1,063 required-gate) in the system’s first two weeks — and continuing operation at the same tempo (16,438 runs by 2026-08-31, 9,661 in the trailing fortnight) — five forensically narrated incidents, including an organic version-skew red that failed closed and loud and a green-while-broken blueprint remediated with input-conditioned behavioral invariants, and transferable

operating lessons: the required-check wedge, pin-lockstep coverage, version-skew honesty as a fleet necessity, and the fixture-vs-live observation boundary.

What we do not claim. The base idea of checking an authored high-level model against extracted source structure belongs to Murphy and Notkin’s reflexion models [15, 16]. Making spec-code divergence a blocking merge condition is independently claimed as a design by SGE [12] and shipped by archgate [3] and SonarQube’s architecture capabilities [21, 22]. Attestation and signing machinery belongs to in-toto and SLSA [18, 24]; architectural fitness functions to Ford et al. [10]. We realize, operationalize, and measure a particular conjunction of mechanisms; we claim none of these underlying ideas.

A note on evidence discipline. Every quantitative claim in this paper is tied to a dated, independently re-readable artifact — a workflow run, an API read, a committed record — and claims we could not verify at read time are absent rather than hedged. Scope limits are stated where the result is reported.

2 Background and Related Work

The closest work. Independently of our work, Grabowski [12] proposed the Spec Growth Engine (SGE; arXiv:2606.27045, submitted 25 June 2026): a machine-readable spec graph with contract/design separation, a context assembler, a vertical-slice growth protocol, and a drift gate making spec-code divergence a blocking merge condition. SGE is the nearest neighbor on the intent-gating axis, and we cite it accordingly and prominently. The differentiation is this: SGE is a design proposal — it is maintained as an internal design-document set with a public release planned, its §7 is a traced walkthrough rather than an evaluation, its gate is a binary block over four hard-error constraint classes with no numeric score, its analysis is static-only, and it specifies no evidence chain and no validation of the checker itself. We report a validated production realization with capabilities SGE does not specify: numeric scoring, a hash-chained evidence chain, served-runtime observation, and measured-recall checker validation. We document the independence timeline via dated artifacts in §7; the priority we claim is realization and measurement, never the idea.

Reflexion lineage. The intent-versus-observed conformance idea is thirty years old: Murphy and Notkin’s software reflexion models [15], extended with Sullivan [16], compute convergences and divergences between an authored high-level model and extracted source. Active descendants include CATMA [6] (microservice deployment-vs-implementation non-conformance, visualization output, no CI gate, not AI-focused), 3Erefactor [14], and ICSA 2025 work on reflexion mapping. bce is a reflexion-model descendant whose distinguishing properties are the gating, evidence, and self-validation machinery around the comparison, not the comparison itself.

Shipping drift gates. archgate [3] is a shipping Apache-2.0 CLI that enforces Architecture Decision Records as executable rules (markdown+YAML with companion `.rules.ts` checks) in a millisecond CI check blocking merges, with explicit AI-agent positioning; it is binary pass/fail, static-only, and publishes no evidence chain, no numeric score, and no checker validation. SonarQube’s Architecture-as-Code [21] shipped in Server 2025.2 with declarative YAML/JSON architecture definitions verified in CI/CD, deviations surfacing as source-line issues; its announced replacement has since landed as a mainline capability — intended-versus-actual architecture with automatically flagged deviations surfacing in the quality gate, announced for SonarQube Cloud in December 2025

and brought to SonarQube Server in 2026.4 (July 2026) alongside an agent-targeted quality-gate profile [22]. Authored-architecture conformance checking in CI is, in other words, now a mainstream commercial capability, and we do not rest any novelty on that axis alone: what this paper claims is the *conjunction* of Table 1 — deterministic numeric scoring, fail-closed required-check gating, hash-chained re-derivable evidence, measured-recall validation of the checker itself, and served-runtime behavioral observation, operating together in production. act101 [1] ships deterministic MERGE/REVIEW/BLOCK verdicts with *signed receipts* for AI refactor/port transformations — the nearest prior art on the signed-evidence axis — but has no authored intent artifact and no architecture score, and its scope is transformation verification only. vFunction [25] provides runtime architectural-drift observability (static plus dynamic) for modernization; it baselines a captured snapshot rather than authored intent, and it notifies rather than gates.

Vacuity and mutation lineages. Our toothlessness gate descends from vacuity detection in temporal model checking: Beer et al. [4, 5] showed how to detect specifications satisfied vacuously — reporting that on the order of 20% of specifications passed vacuously in initial industrial verification runs — and Kupferman and Vardi generalized the theory [13]. We claim only the CI-time application of this lineage to architecture blueprints, not the concept. The nearest kin to our checker validation is mutation testing, but aimed at a different object: Farrag [9] uses mutation scores to validate *generated-code test suites*; we seed defects to validate the *drift checker itself* (§4.4).

Adjacent lines. Upstream spec-first tooling (GitHub Spec Kit, Kiro, Tessl [2, 11, 23]) supplies specifications as generation input, and some of it does re-read the built system afterwards: Spec Kit ships a **converge** command that assesses the current codebase against the feature’s spec, plan and tasks and appends the remaining work. What that assessment produces is spec/plan/task consistency feeding agent-assisted completion — not a fail-closed, independently scored verdict over an *extracted architecture graph*, gated in CI, with a vacuity oracle standing behind it. That verdict is the axis we claim; we do not claim these tools ignore the built system. The same ecosystem is meanwhile growing conformance-adjacent gating from the community side: Blueprint Index [17], a Spec Kit community-extension submission (July 2026), maintains an agent-authored architecture map with a tiered CI gate that blocks on critical map-versus-code contradictions — by its author’s own scope statement “detection, not conformance,” with no score, no evidence chain, and no checker validation. And the problem class itself is being named in the literature: Casserini et al. [7] describe *agentic entropy* — systemic drift between agentic actions and intended architecture — in a position paper proposing a process-oriented explainability framework, with no implemented gating system. We read both as concurrent adjacent work: evidence the problem is real and moving, combining none of the mechanisms this paper measures. The agent-runtime governance line — AgentSpec, MI9, agent behavioral contracts, DEMM-Bench [20, 26, 27] — constrains *agent behavior* at runtime, not built-system-versus-blueprint conformance. Piskala [19] owns the spec-rigor taxonomy and Farrag [9] the specification-governance framing; we adopt neither as a contribution. Finally, in-toto and SLSA [18, 24] own attestation machinery for supply-chain provenance; our sweep surfaced no prior application of hash-chained, content-addressed evidence to architecture-conformance verdicts — an absence-of-evidence result, stated as such.

Table 1 renders the differentiation compactly; it is the spine of this section.

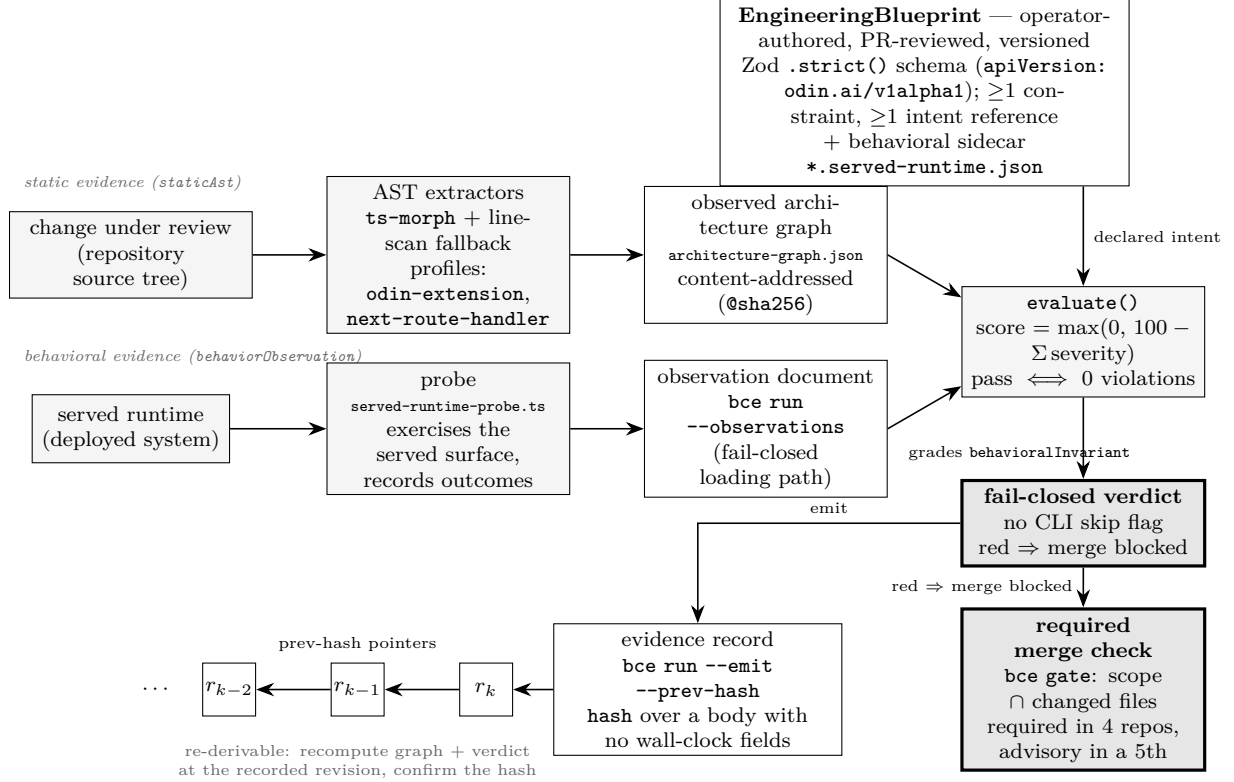


Figure 1: The bce loop (§3). An operator-authored, PR-reviewed **EngineeringBlueprint** (with an optional `*.served-runtime.json` behavioral sidecar) declares intended architecture. AST extractors (`ts-morph` with a line-scan fallback, organized as extraction profiles) build the observed architecture graph from the change under review, while a served-runtime probe (`served-runtime-probe.ts`) records an observation document from the deployed system, graded through the same machinery via `bce run --observations`. `evaluate()` grades the declared-vs-observed difference deterministically ($\text{score} = \max(0, 100 - \sum \text{severity})$; pass requires zero violations) into a fail-closed verdict — within an evaluated blueprint there is no CLI skip flag; the zero-discovered floor is asserted by the periodic sweep mode of §6.5, not by the per-PR verb — enforced by `bce gate` as a required merge check in four repositories (advisory in a fifth). Each verdict emits a hash-chained, content-addressed evidence record (`bce run --emit --prev-hash`); records are re-derivable: a third party at the recorded revision can recompute both graph and verdict and confirm the hash.

3 The bce System

3.1 A conformance engine, not a compiler

bce is a deterministic policy-evaluation pass over an extracted architecture graph, emitting a graded, evidenced verdict (Figure 1). It generates nothing and transforms nothing; it measures. The intent artifact is an **EngineeringBlueprint**: an operator-authored, versioned, PR-reviewed JSON document validated by a strict schema (`Zod.strict()`, `apiVersion: odin.ai/v1alpha1`) that requires at least one constraint and at least one intent reference — a blueprint must bind to stated intent and must assert something. Every report also carries an explicit coverage envelope (`coverage.unsupported`) naming constructs the extractors could not analyze, so unchecked surface is declared rather than silently passed. Constraint semantics evolve under a widen-only ratchet: schema revisions may add expressive power but never silently relax what an existing constraint enforces.

3.2 Constraint taxonomy and extraction profiles

Blueprints draw on eleven constraint types: `forbiddenDependency`, `requiredDependency`, `requiredComponent`, `forbiddenPath`, `forbiddenFile`, `forbiddenEgress`, `requiredEvidence`, `minimumMetric`, `customPolicy`, `behavioralInvariant`, and `forbiddenPattern` (with ReDoS-safe matching); Table 2 gives the full taxonomy. Severity is author-declared per constraint (`info` through `critical`), and each violation deducts the corresponding severity weight of §3.3 — there are no per-type severity defaults. Eight of the eleven types are enforcing in the current engine; the remaining three are schema-reserved and evaluate as explicit skips, recorded in the report summary — never a silent pass (engine 0.13.0, published 2026-08-09, tightens this further: an ungradeable constraint is refused per-constraint, fail-closed, rather than skipped-with-note; the fleet’s gates began that deliberate migration on 2026-08-09 and substantially converged by 2026-08-12 — §6.1, addendum, reports the migration and what the nightly monitoring caught mid-flight). The observed graph is built by AST-level extractors (`ts-morph`; TypeScript/Node-centric) with a line-scan fallback, organized as pluggable extraction profiles — the two production profiles used in this paper (`odin-extension`, `next-route-handler`) are severable by design from the engine core. The engine’s runtime dependency surface is deliberately small: `zod` and `ts-morph` only, with no internal imports.

3.3 Scoring and verdict semantics

Evaluation grades the difference between declared and observed architecture as **score = max(0, 100 − Σ severity weights)**, with weights of 40 (critical), 20 (high), 10 (medium), 5 (low), and 0 (info) per violation. The verdict is not a threshold on the score: **pass requires zero violations**; the score grades the magnitude of failure. Semantics are fail-closed — an empty scan or a blueprint with zero enforcing constraints yields *fail*, never a vacuous pass. Output is byte-deterministic across runs, and every report carries a content-addressed `evidenceRef` (e.g., `architecture-graph.json@sha256:...`) binding the verdict to the exact graph it graded.

3.4 The fail-closed gate verb

bce gate is the CI entry point: it discovers blueprints under `.ai/blueprints/*`. `engineering-blueprint.json` (the open-core release of §10 defaults to `.blueprints/` and the `*.blueprint.json` suffix), intersects each blueprint’s scope with the change’s files,

and evaluates the intersecting set. Within an evaluated blueprint the semantics are fail-closed and **there is no CLI skip flag**: a malformed blueprint, an engine below the blueprint’s `minEngineVersion`, a scan below the minimum-file floor, and a blueprint with zero enforcing constraints each *fail* the run rather than skipping it. Two limits of that guarantee are properties of *scoping* rather than of grading, and we state them plainly because a reader can check them in a minute: a change intersecting no blueprint’s scope is reported as `0 blueprint(s) evaluated` and exits 0 — it was correctly graded against everything that applied to it. In the engine version these measurements were taken under, a repository with *no blueprints at all* also exited 0, because the verb’s exit code was a function of the reports it produced and an empty report set carries no failure; the floor that made a *vanished* portfolio fail lived not in the verb but in the workflow around it, and still does — the blueprint-conformance gate workflow in each of the five repositories runs a periodic full-sweep mode that asserts a non-zero discovered count and fails when it reaches zero. We report that as measured, and we also fixed it: the open-core release of §10 refuses a zero-*discovery* run outright, exiting 2 (its structural-refusal code, distinct from 1 for a graded red) with `0 blueprint(s) discovered -- a repository that gates nothing has proven nothing`. The distinction that matters is discovery versus selection: zero *selected* is a legitimate green, zero *discovered* is a gate that could be retired with `rm`. A green per-PR gate certifies that every blueprint whose scope the change touched was found, graded, and satisfied; the assurance that some blueprint applies at all is supplied by the sweep, not by the per-PR verb.

3.5 The evidence chain

`bce run --emit --prev-hash` appends an evidence record to a hash chain. Each record’s `hash` commits to a canonical body that carries no wall-clock fields at all — they are excluded by construction rather than stripped after the fact, which is the stronger property — so the hash commits to the conformance-relevant content and nothing else; records are content-addressed and **re-derivable** — a third party with the repository at the recorded revision can recompute both the graph and the verdict and confirm the hash. In production, the four committed evidence records on the platform monorepo’s main branch carry chain depths of **20, 17, 14, and 12 links** (measured as the commit history of each record file); the chains have not grown past their 2026-07-23 links, for a disclosed operational reason reported in §6.5. Re-derivability is demonstrated nightly rather than merely claimed: an independent, hermetic re-implementation — the nightly sweep’s trust rollup — recomputes every record’s hash from the stable serialization of the record minus its identity fields and commits the verdict. We apply attestation *practice* here; the machinery lineage belongs to in-toto and SLSA [18, 24].

3.6 The toothlessness gate

The C-AUTH-02 flip of §1 shows that the dangerous blueprint is not the one that fails but the one that cannot fail. `bce teeth` assesses this directly — it is the subject of §4.1, where the mechanism, its portfolio audit, and its CI wiring are reported together.

3.7 Served-runtime behavioral observations

Static extraction cannot see whether the *running* system behaves as intended. `bce run --observations <path>` (shipped in engine 0.9.0 and hardened in 0.11.0) grades `behavioralInvariant` constraints against an observation document produced by a probe exercising the deployed runtime. Behavioral teeth live in sidecar `*.served-runtime.json`

blueprints, separating the behavioral gate from the structural gate so each can be pinned, evolved, and required independently. §5 reports the design; §6.2 the operational proof.

3.8 Honest version-skew semantics

A fleet does not upgrade atomically, so the engine (18 published versions, latest 0.17.1, as of 2026-08-31; the 0.16.x minor was never published — the version sequence steps from 0.15.0 directly to 0.17.0, and we say so rather than leave the gap to read as an error) treats version skew as a first-class honesty problem. A constraint type unknown to the evaluating engine yields an **explicit advisory, never a silent pass**; blueprints may declare `minEngineVersion`, producing a clear error rather than degraded evaluation; and when the gate runs in a mode that skips a constraint class, the skip is surfaced as an advisory in the report. The invariant throughout: skew may reduce what is checked, but never silently, and never in the direction of a false green. This thesis received an unplanned production demonstration on 2026-08-01, when a stale engine leg met newer constraint vocabulary and failed closed and loud rather than silently green (§6.5, Incident 1).

3.9 Propose-not-apply remediation

Remediation is deliberately asymmetric to detection. `bce apply` is human-attended: dry-run by default, requiring `--confirm` to write, and requiring `--dangerous-ack` with a recorded `--rationale` for any edit that would *relax* a constraint. The engine proposes; a human applies. An autonomous loop may thus tighten its own leash but can never quietly loosen it — the same fail-closed asymmetry that governs the verdict governs the remediation path.

4 Validating the Checker Itself

A conformance gate is a claim about claims: it asserts that a green verdict means the system was measured against its authored intent and found conformant. That assertion fails silently in two ways. A *blueprint* can be unable to fail — every constraint vacuously satisfied by any input graph — so that a perfect score certifies nothing. And the *checker* can be unable to catch — a defect class the evaluator never flags — so that even a toothed blueprint yields unearned green. The first failure mode is the toothlessness problem one level down; the second is the same problem one level up. `bce` operationalizes a defense at both levels, and this section reports both: a gate that refuses unenforceable blueprints, and a measured-recall protocol that treats the engine itself as the system under test.

4.1 The Toothlessness Gate: Refusing Blueprints That Cannot Fail

The `bce teeth` verb realizes, at CI time, the concern that the model-checking community named *vacuity*: a specification satisfied for degenerate reasons certifies nothing about the artifact [4, 13]. The concern is not hypothetical at industrial scale — Beer et al. report that roughly 20% of specifications passed vacuously in initial verification runs at IBM Haifa [5]. We claim only the application of this lineage to blueprint constraint-sets inside a CI gate, not the concept.

The mechanism uses the evaluator as its own oracle. For a candidate blueprint, `assessTeeth()` synthesizes realistic mutations of the extracted architecture graph — the kinds of change a defective pull request would actually make — and re-runs `evaluate()` over each mutant. The mutation catalogue is per-constraint-type and minimal: for required-shape constraints the mutation *removes*

what satisfies them (delete every satisfying *guards* edge of a `requiredDependency`; delete every component of a `requiredComponent`’s target type), and for forbidden-shape constraints it *adds* exactly the drift a bad pull request would introduce (a forbidden import edge for `forbiddenDependency`, an egress edge to a blocked provider host for `forbiddenEgress`, a file under the banned path or glob for `forbiddenPath/forbiddenFile`, a matching content line for `forbiddenPattern`, a constant-output flattening of the observed behavior for `behavioralInvariant`). For example, removing the satisfying *guards* edge from one route-handler component reddens a C-AUTH-02-class authentication constraint, and that mutation is recorded as the constraint’s witness. A constraint has teeth if and only if some realistic mutation flips the verdict from green to red; a blueprint that stays green under every mutation is graded `TRIVIALY_GREEN` and refused. The third verdict is the honesty valve: the default posture is `INDETERMINATE`, reported whenever no discriminating mutation can be synthesized for a constraint. An unprovable tooth is surfaced as unproven, never presumed enforceable — the gate does not manufacture confidence it did not earn.

The gate itself is validated the same way everything else in this paper is: against real fixtures, not synthetic toys. Beyond its 11 unit tests, the shipped implementation (local-agents PR #309, merged 2026-07-28) carries two end-to-end proofs on production-derived material: the `luna-chat-extension` blueprint — the same surface that anchors the recall corpus of §4.2 — is graded `TOOTHED` against its real extracted graph, and a variant of the same blueprint stripped of its enforcing constraints is rejected as toothless. The gate can thus be shown to both accept an enforceable blueprint and refuse its defanged twin over the same codebase.

Beyond the accept/refuse pair, we measured the oracle over the full production portfolio. On 2026-08-01/02 we ran `bce teeth at engine 0.12.0` — the version pinned by the agent-runtime, control-plane and engine repositories’ gates at the time; the platform monorepo’s five workflows were on the 0.12.1 patch and the advisory fifth repository on 0.6.0, so this is not a single fleet-wide pin (see footnote 1) — over all 33 CI-gated fleet blueprints across the five repositories of §6.1, each scanned against a shallow clone pinned to its repository’s default-branch head. Table 3 gives the distribution.

Oracle-version caveat (load-bearing, not a footnote). Every figure in Table 3 — including the 212-of-217 witness count — is a **0.12.0 measurement**, and the oracle has since changed along exactly the axis the table reports. Engine 0.14.0 (published 2026-08-12; the fleet has since moved onto it, so no gate still runs the 0.12.0 oracle) carries a teeth channel-availability fix: a constraint that is *structurally incapable of firing* is no longer certified as toothed. That fix reclassifies by construction, and we have measured it doing so: re-running `bce teeth` at 0.14.0 against the control-tower portfolio moves `control-tower-ontology` from 3-of-3 toothed to 2-of-3 — one constraint in three, on that blueprint, was previously certified as having teeth it did not have. No blueprint we re-measured became *toothless*, so no gate flips, but the witness-level distribution in Table 3 would move under the newer oracle by an amount we have not measured fleet-wide. We therefore report the 0.12.0 numbers as what they are — a dated measurement under a superseded oracle — rather than restating them as current. A fleet-wide re-run at 0.14.0 is the obvious follow-up, and until it exists the honest reading of $\approx 0\%$ vacuity is that it is a lower bound on vacuity as the *newer* oracle would count it.

That follow-up has since been run — and it moved the oracle’s vocabulary, not only its counts, in exactly the direction this caveat predicted. The pre-0.17 oracle turned out to *free-tooth* every `forbiddenPattern` constraint: its reddening mutation injected a synthetic pattern hit into the observed graph and then “detected” it, so even a constraint carrying the unmatchable regex `(?!)` graded toothed (the canary measurement of 2026-08-23 that motivated the fix). Engine 0.17.0 (pub-

lished 2026-08-23; the version every platform-monorepo gate literal pins as of that week) splits the verdict honestly into extractor-real TOOTHED, EVALUATOR_REFUTABLE (the evaluator flips when handed the violating fact, but a realistic extractor-level path to planting it was not established — honest, exit 0, deliberately not lumped with vacuity), and PATH_SELECTS_NOTHING (a constraint whose path glob selects zero tracked files — the dead-glob class, now named by the engine itself). Re-measured on 2026-08-24 over the platform monorepo’s since-grown portfolio (28 blueprints, 180 constraints) at 0.17.0: at the blueprint level **9 toothed, 19 evaluator-refutable, 0 toothless**; at the constraint-witness level **31 / 144 / 4 path-selects-nothing / 1 indeterminate**. An A/B sweep of 0.14.0 against 0.17.0 measured **zero exit-code flips across all 112 pairs** — the regrade changes teeth vocabulary, never gate outcomes, and the gates branch on exit code rather than verdict string by pinned design. The honest reading after the regrade: the earlier blanket TOOTHED over-counted extractor-real teeth by exactly the free-toothed class, and the oracle now reports that over-count itself — the checker-validation discipline this paper argues for, applied to the checker’s own vacuity oracle.

Every fleet blueprint has at least one constraint a realistic graph mutation reddens. At the finer per-constraint witness level, exactly one constraint of 217 is trivially green — a **forbiddenDependency** whose relative-specifier frame can match no edge the extractor produces, i.e. a genuine authored vacuity the oracle found — and four are **INDETERMINATE**: all four are **customPolicy** declarations the evaluator does not yet enforce, which the oracle quarantines as unwitnessable rather than counting green. No portfolio constraint fires on the current main trees: the portfolio is simultaneously *green* (conformant) and *toothed* (falsifiable). The oracle also proved runnable outside its CI context: 48 of the 50 committed blueprint files fleet-wide (96%), including 100% of the gated portfolio, re-run from nothing but a pinned shallow clone; the two exceptions are structural, not tooling failures (one blueprint targets a repository that is not on the public forge, and one compile-harness fixture’s scope matches zero files in the live tree, refused by the fail-closed floor exactly as designed).

Against the vacuity literature’s report that on the order of 20% of specifications pass vacuously when a detector is first pointed at a mature suite [5], the measured $\approx 0\%$ is *not* evidence that blueprint authors avoid vacuity unaided. The oracle and the engine’s fail-closed authoring floors were in the authoring loop for most of this portfolio — the authoring verb refuses a scope matching zero files, minimum-file floors kill empty scans, and a non-compiling regex is a schema-level error — and at least one portfolio blueprint is on record hitting the score-0 fiction trap during authoring and being re-framed until it had teeth. The claim this measurement supports is that **a mutation-based vacuity oracle in the authoring loop drives fleet vacuity to approximately zero**, against a literature baseline describing suites audited without one. Two further disclosures: the census set is the *gated* portfolio, so blueprints that never grew teeth may simply never have been promoted into it (the ungated tier of committed blueprints shows the same ≈ 0 trivially-green rate, which weakens but does not eliminate this selection concern); and the **INDETERMINATE** class is a disclosure, not noise — it is the portfolio’s residual declared-but-unenforced surface, named per constraint. At measurement time no CI workflow invoked **bce teeth**; the numbers above are a portfolio audit by the shipped verb. Immediately after the measurement (2026-08-01), the oracle was wired into the blueprint-conformance gates of two of the four required-check repositories — the platform monorepo and the agent-runtime repository — so a portfolio blueprint changed by a pull request must now prove its own teeth to merge there: a blueprint-level **toothless** verdict fails the existing required check, while constraint-level **TRIVIALY_GREEN/INDETERMINATE** surface as warnings without failing — a deliberate introduction posture that may only tighten. The wiring subsequently completed: as of 2026-08-10 the oracle runs inside the conformance gates of four of the (now five) required-check repositories (§6.1, addendum; §8).

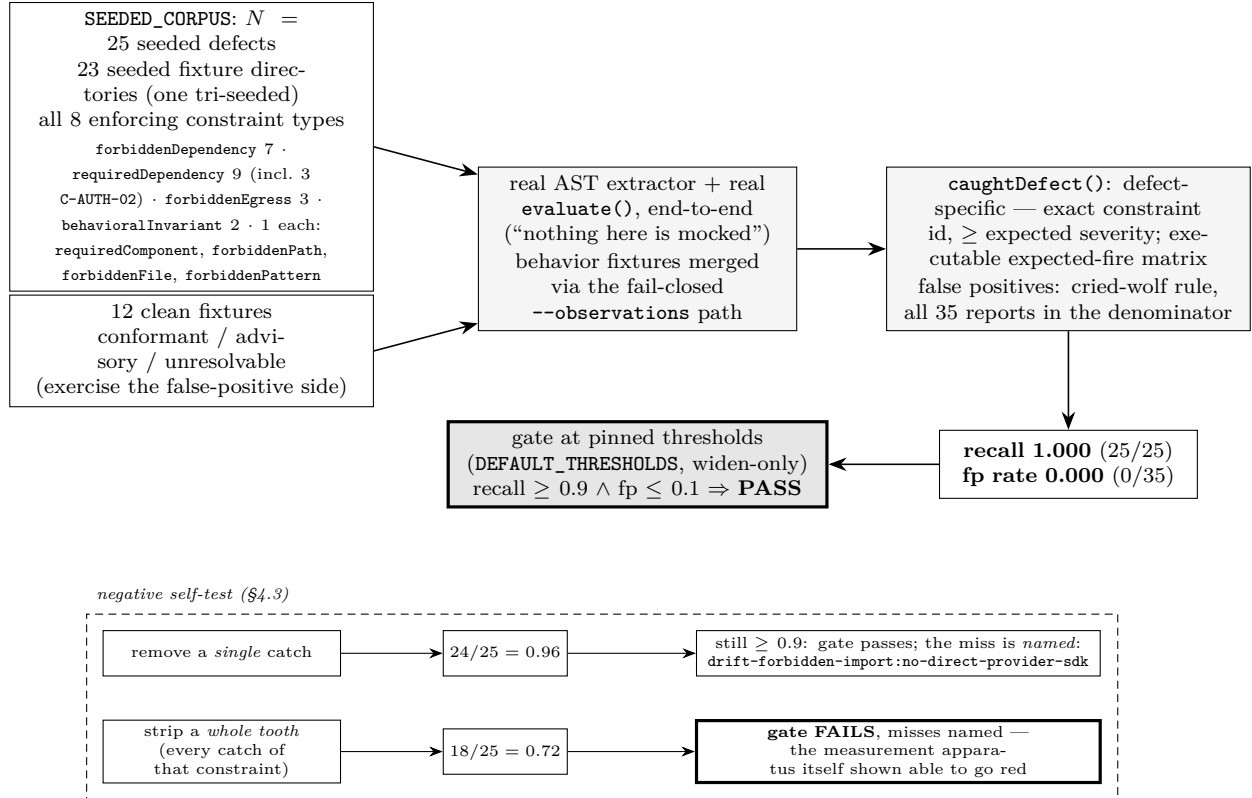


Figure 2: The checker-validation harness (§4.2–§4.3). $N = 25$ conformance defects (SEEDDED_CORPUS) seeded into 23 fixture directories derived from production extraction surfaces — covering all eight enforcing constraint types, including three defects in the motivating C-AUTH-02 authentication-coverage class of §1 — plus 12 clean fixtures are evaluated end-to-end by the real extractor and evaluator. Catches must be defect-specific (caughtDefect()): exact constraint identifier at or above the expected severity, with an executable expected-fire matrix asserting exact violation multisets), and any violation a fixture did not seed is a false positive over the full 35-report denominator. Measured: recall 1.000 (25/25) at a false-positive rate of 0.000 (0/35), a passing verdict at the pinned, widen-only thresholds (recall ≥ 0.9 , fp ≤ 0.1 ; as of 2026-08-02, local-agents PR #345). Inset: the negative self-test — removing a single catch yields 24/25 = 0.96 (the gate still passes and the miss is named); stripping a whole tooth yields 18/25 = 0.72 and the gate fails, naming the misses.

4.2 The Measured-Recall Protocol

The toothlessness gate asks whether a blueprint *could* fail. The recall protocol asks whether the checker *does* catch: we seed known conformance defects and measure the fraction the engine refuses (Figure 2). The corpus is a set of $N = 25$ seeded defects (SEEDDED_CORPUS; the nine original entries byte-frozen, sixteen appended — as of 2026-08-02, local-agents PR #345, merge f8d6a900), embedded in fixture corpora spanning four surfaces derived from the engine’s production extraction and observation profiles, each carrying its real authored blueprint:

- **Extension surface** (luna-chat-extension@0.1.0, the odin-extension extraction profile; 14 defects): the six originals — direct import of a provider SDK bypassing the governed gateway, provider reach via re-export and via dynamic import(), tool registration that skips

the governed registration path, and two decoy variants (a shadow-harness decoy and a decoy-registration fixture) that *simulate* compliance while violating it — plus CommonJS `require` and template-literal-specifier variants of provider reach, two further ungoverned-registration variants, and a tri-seeded unrecognizable-factory fixture whose single directory carries three defects (an unrecognizable extension factory, a forbidden provider import, and a zero-targets fail-closed arm of the registration constraint).

- **Egress surface** (`egress-reader@0.1.0`; 3 defects): three idiomatic variants of raw provider-host egress — a house-idiom HTTP call, an `undici` client, and an options-bag construction — each evading the governed-hosts allowlist by a different syntactic route.
- **Route surface** (`route-guard`, the `next-route-handler` extraction profile; 6 defects): three authentication-coverage defects in the motivating C-AUTH-02/tenant-guard class of §1 — a route handler missing its guard, a new route file with unguarded handlers, and a decoy guard object that *names* the guard symbol without calling it — plus a route under a forbidden legacy path, a shadow provisioner file that mints no components, and a mocked-metric pattern planted at an exact file and line.
- **Behavior surface** (`served-behavior`; 2 defects): a constant-function violation and a property-oracle violation of a `behavioralInvariant`. These two entries measure the engine’s observation-grading arm over *synthetic* observation documents — committed fixtures hand-authored to the `behaviorObservation` shape, not recordings of a running system — merged through the real fail-closed `--observations` path; neither the live probe (stimulus → served runtime → hash recording) nor the authenticity of the observation document is inside the fixture corpus’s measurement scope.

Coverage spans **all eight enforcing constraint types**: `forbiddenDependency` (7 defects), `requiredDependency` (9 — six governed-registration defects including a zero-targets fail-closed arm, plus the three C-AUTH-02 tenant-guard defects), `forbiddenEgress` (3), `behavioralInvariant` (2), and one each of `requiredComponent`, `forbiddenPath`, `forbiddenFile`, and `forbiddenPattern`. In particular the motivating C-AUTH-02 class of §1, unrepresented in the original nine-defect corpus, is now covered by three defects — the validation exercises the constraint class that motivated the system.

Each defect is a realistic drifted variant of its surface: a complete fixture directory containing exactly the violation a plausible bad pull request would introduce, drawn from an on-disk fixture library of 35 fixture directories — 16 extension-surface, 7 egress-surface, 9 route-surface, and 3 behavior-surface variants, as of 2026-08-02 — of which 23 distinct directories seed the 25-defect corpus (the tri-seeded fixture carries three defects) and 12 are clean; the clean set spans conformant, advisory, and unresolvable classes that exercise the false-positive side of the gate. The fixtures are seeded variants derived from production surfaces; they are not live production routes.

Three rules keep the resulting numbers meaningful:

Catches are defect-specific, never blanket. `caughtDefect(defect, report)` requires a violation carrying the defect’s exact constraint identifier at or above its expected severity — a security defect flagged only at `info` is not caught. The expanded corpus raises the bar further with an **executable expected-fire matrix**: for every seeded route and behavior fixture, the harness asserts the report’s exact violation multiset (constraint identifier and component), so same-constraint over-flagging inside a seeded fixture is a measured fact rather than an invisible possibility. At the pipeline level the bar is higher still: a catch requires exit code 2, a red summary, *and* the constraint

identifier present in the failure list. A checker that reddened everything would score zero, not twenty-five.

False positives are cried-wolf events. Any violation raised for a constraint its fixture did not seed counts against the checker, and the twelve clean fixtures must each produce a green, zero-violation verdict and a proven-pass report carrying the `coverage.unsupported` coverage envelope; every report — 35 in all, 23 seeded-fixture reports plus 12 clean — enters the false-positive denominator. This is what keeps the recall number honest: a panel that refuses everything trivially achieves recall 1.0, so the false-positive ceiling is the half of the gate that makes the other half worth reporting.

Thresholds are pinned and widen-only. The gate passes iff $\text{recall} \geq 0.9$ and false-positive rate ≤ 0.1 (`DEFAULT_THRESHOLDS`); the thresholds are governed as a pinned surface that may only ever tighten. A measured failure is an honest fail that keeps the engine advisory — never a licence to lower the floor. The gate additionally refuses degenerate configurations outright ($\text{minRecall} \leq 0$ or $\text{maxFpRate} \geq 1$ yield a governance-refused verdict), and the corpus itself is hardened against drift: growth is append-only with the original entries byte-frozen, a CI floor on the corpus size (raised with the expansion) turns silent shrinkage into a build failure, a CI test asserts every fixture resolves on disk (a dangling reference fails the build rather than silently shrinking the denominator), and an empty corpus measures recall as honestly 0, never vacuously 1.

4.3 Results

At the **engine level** — the altitude at which the expanded corpus is measured — an end-to-end proof test runs the real AST extractor and the real `evaluate()` pass over the real fixture directories (the test’s own comment is “Nothing here is mocked”), merging the behavior fixtures’ recorded observations through the real fail-closed `--observations` loading path, and asserts recall exactly 1 with all twenty-five defects caught: **recall 1.000 (25/25, every catch defect-specific, with the expected-fire matrix asserting exact violation multisets)** at a false-positive rate of **0.000 (0 false positives over 35 reports: 23 seeded fixtures + 12 clean)**, and a passing gate verdict at the pinned thresholds — untouched at 0.9/0.1 through the expansion (as of 2026-08-02, local-agents PR #345, merge f8d6a900; full package suite 41 files / 559 tests green). At the **pipeline level**, the original nine-defect corpus had earlier been run end-to-end through the repository’s blueprint-acceptance-check binding (odin-labs PR #763, merged 2026-07-21): measured recall 1.0000, all catches defect-specific, at a false-positive rate of 0.0000, with byte-identical output across two independent runs. The expanded-corpus figures reported in this paper are the engine-level measurement.

The protocol also carries a negative self-test with two arms (Figure 2, inset), and at $N=25$ the arms discriminate between an honest miss and a gate failure. Removing a **single** catch yields $24/25 = 0.96$ — the gate still clears the 0.9 floor, and the suite asserts the miss is *named* (`drift-forbidden-import:no-direct-provider-sdk`) rather than silently absorbed. Stripping a **whole tooth** — every catch of the `no-direct-provider-sdk` constraint — yields $18/25 = 0.72$, and the gate verdict fails, naming the misses. The measurement apparatus is thus itself shown able to go red — the toothlessness principle applied to the recall gate.

Twenty-five seeded defects and twelve clean fixtures is still a modest corpus, and a perfect score over it is evidence of checker competence on these defect classes, not a general recall guarantee. (At these sample sizes a 95% Wilson interval places the lower bound on recall at 0.86 for 25/25 and 0.89 for 34/34 (bounds floored, never rounded up), and the upper bound on the false-positive

rate at 0.099 for 0-of-35 reports and 0.079 for 0-of-45; we report these to size the corpus honestly, not to claim behavior beyond it.) Coverage, however, is no longer the sharper limit: the twenty-five defects span **all eight enforcing constraint types**, and the motivating **C-AUTH-02** class of §1 (**requiredDependency** authentication coverage), unrepresented in the original corpus, is now exercised by three defects — including a decoy guard that names the guard symbol without calling it. The remaining scope limits are stated here and in §7: every defect was seeded by the paper’s author rather than harvested from an external deployment, and the two **behavioralInvariant** entries measure the engine’s observation-grading arm over *synthetic* observation documents — committed fixtures hand-authored to the **behaviorObservation** shape, not recordings of a running system — merged through the real fail-closed `--observations` path; neither the live probe (stimulus $\rightarrow$ served runtime $\rightarrow$ hash recording) nor the authenticity of the observation document is inside the fixture corpus’s measurement scope. The rigor story remains the construction — defect-specific matching with an executable expected-fire matrix, the cried-wolf rule over a 35-report denominator, on-disk corpus integrity enforced in CI, byte-determinism, and pinned widen-only thresholds. Growing the corpus further — more clean fixtures, and ultimately defect classes harvested from external pilot deployments rather than the author’s own surfaces — is future work (§8).

Corpus growth and re-measurement (2026-08-10). The corpus is live machinery, not a frozen table, and it has since grown — append-only, the originals byte-frozen — to **N = 34** seeded defects (32 distinct seeded fixtures, 13 clean fixtures, 45 reports in the recall run). The nine additions comprise **five Python-surface defects** (direct, aliased, and parenthesized provider imports; a committed-secrets module; a hardcoded provider key) behind a new pluggable extraction-profile registry whose **PythonImportExtractor** documents its undetected constructs honestly — `__import__` and `importlib` are each pinned by a test asserting the construct is genuinely *not* detected — and four TypeScript syntactic-bypass variants (`import-equals`, `subpath-prefix imports`, `new URL()` `constant-hops`, `globalThis.fetch`). Re-running the protocol on 2026-08-10 in a fresh clone reproduces **recall 1.000 at false-positive rate 0.000 over 45 reports** (full suite 645/645 at the measured revision). The corpus additionally carries a standing **mutation hunt**: eight deterministic drift mutations of the clean baseline trees — each the syntactic shape of a plausible bad pull request, including a Python arm — must each redden the engine, with two documented non-flags (a type-only import; a commented-out Python import) required to stay green; at the 2026-08-10 measurement **zero of the eight mutants survive**, and the corpus-size floor ratchets tighten-only. This growth lives in the open-core lineage of §10 — a genericized, namespace-migrated protocol reproduction of the internal corpus (nine paper-baseline entries plus twenty-five expansion entries), not a byte-identical copy; the N=25 figures above remain the internal engine’s frozen 2026-08-02 measurement, and both measurements are dated and separately citable.

4.4 Relation to Mutation Testing

The nearest methodological kin is mutation testing, and the distinction is the object under test. Mutation testing validates *test suites* by seeding faults into the program; in the AI-codegen literature it has been applied to validate generated-code tests (§2). Our protocol seeds *conformance defects* into fixture corpora to validate the *drift checker itself* — the component whose verdicts the whole gating chain depends on. Likewise, the toothlessness gate’s mutation of the architecture graph is not program mutation: the mutant is the observed-architecture input, and the oracle is the evaluator whose enforceability is in question. To our knowledge, published architecture-conformance tools do not report a measured validation of their own checker (§2, Table 1); we regard reporting one — including its small N — as part of the experience this paper exists to transmit.

5 Served-Runtime Behavioral Conformance

5.1 Observations as First-Class Conformance Evidence

Everything graded so far is static: extractors read source, and the evaluator grades the extracted graph. But a class of intent survives static extraction untouched — *behavioral* invariants of the served system, such as whether two concurrently attached sessions can be steered independently, or whether a detach–remount cycle is idempotent. bce operationalizes these as sidecar blueprints (`*.served-runtime.json`) whose constraints are behavioral invariants, graded by `bce run --observations <path>` (a 0.9.0 feature, hardened in 0.11.0) against a recorded observation document rather than an extracted graph. The observations are produced by a probe (`served-runtime-probe.ts`, `odin-suite` PR #1002) that exercises the served runtime — the agent’s terminal session-management surface (`agent-tui-cockpit`) — and records what the running system actually did. Grading then proceeds through the same deterministic scoring and fail-closed verdict machinery as §3: the seam that changes is the evidence source, not the semantics.

This split — structural constraints in the per-PR static gate, behavioral teeth in a sidecar graded from runtime observations — is deliberate. Behavioral properties cannot be established by reading source, and forcing them through a static gate either mis-grades them or, worse, passes them vacuously. Post-split, the subject carries **17 teeth: 7 behavioral (served-runtime blueprint 0.3.0) and 10 structural (engineering-blueprint 0.4.0)**; the behavioral seven include multi-session independent steering, create-many-then-list, detach–remount idempotence, and inspect-without-attach. As a concrete instance: the multi-session steering invariant requires that two concurrently attached sessions given distinct steering stimuli produce distinct observed outputs — a constant or replayed response fails the property oracle (the constant-function/mock detector of Table 2). The operational record of this gate — its promotion to a required check, its green and red proofs, and its flake record — is reported in §6.2.

5.2 Why Runtime Observations Close the Anti-Mock Gap

Static conformance grades what the source *promises*; a sufficiently determined defect — or a sufficiently convincing mock — can keep every structural promise while the served behavior regresses. The decoy fixtures of §4.2 are the static shadow of this problem: code that performs compliance. Served-runtime observations close the gap from the other side. The probe does not read the code; it drives the running surface and records outcomes, so — *given observations faithfully recorded by the probe* — a behavioral tooth is satisfiable only by a system that actually behaves, and a test double that fakes the structural graph earns nothing here. That conditional is load-bearing and we make it explicit: the engine grades the observation document it is handed and does not authenticate its provenance, a limitation we state in §7 rather than a guarantee we claim. Combined with the toothlessness gate below it and the measured-recall validation beneath that, the behavioral gate completes the measurement stack this paper reports: intent that can fail, a checker shown to catch, and a runtime that is graded on what it does when served rather than on what it claims when read.

5.3 Input-conditioned behavioral invariants in standing operation (2026-08-10 addendum)

Since the data freeze, a second production subject has carried the behavioral machinery through a full remediation arc that the incident record of §6.5 (Incident 4) narrates. An operator-steering bridge subsystem on the platform monorepo, blueprinted at v0.4.0 with ten static constraints,

scored 100/*pass* while its inbound leg was operationally broken — structurally toothed, behaviorally vacuous. The remediation added four *input-conditioned behavioralInvariant* constraints (on-main identifiers C13–C16 at blueprint v0.7.0, 14 constraints total): inbound-goal reply, unattended hands-off round-trip, terminal-delivery read-back, and terminal provenance-at-source — each proven red-before/green-after against the *real* producer rather than a synthetic stub (the C13 proof arc: 100 at v0.4.0 → 60/fail with the invariant present but zero observations, fail-closed → 100/pass on live-consumer observations → 20/fail under a seeded output regression caught by the constant-function oracle → 60/fail when an observation is dropped below the two-stimulus floor; odin-labs PRs #1183, #1235, #1227). A steward re-grade script regenerates all four observation sets from the running system and re-grades fail-closed (exit 2 on any red); the *mechanism* is proven — three green runs on 2026-08-09, score 100 at pinned engine 0.12.1, with the fail-closed leg observed firing during development — and the standing daily cadence has since been demonstrated over a multi-day series: five green runs across three consecutive days (2026-08-09 through 2026-08-11), each verdict green at score 100 with the same observation vector, the daily firings driven by an installed operator-host scheduler (the audit trail is operator-host, not repository-committed, and we cite it as such). The same blueprint meanwhile exhibits a deliberate *grading split* that the fleet’s evidence ledger now records explicitly: the CI-side evidence sink, which grades without runtime observations, honestly records the blueprint at score 0/*fail* — exactly the four behavioral invariants violated, fail-closed absent observations — at every recorded revision, while the runtime steward with freshly regenerated observations grades 100/*pass*. One blueprint, two honest surfaces: the static gate refuses to certify behavior it cannot observe, and the surface that can observe it is the one whose green counts. The boundary this machinery does *not* cross is reported as Incident 5.

6 Production Experience

bce is not an evaluated prototype; it is the standing merge control on the fleet that builds it. What follows is **early production experience**, deliberately labeled as such: the first blueprint-workflow run in the estate was 2026-07-19, and the behavioral gate was promoted to a required check on 2026-07-30. Unless noted otherwise, all operational figures in this section were read from the live GitHub API at the 2026-08-01/02 data freeze; figures explicitly dated 2026-08-10 (the topology and continuation addenda, and Incidents 4–5) were read the same way at that later date. Thirteen days is a short window by calendar; the unit of analysis that matters here is events, not days. The estate under gating is an agent fleet whose pull-request tempo compresses experience-per-day far above a human-paced shop: the required gates and scheduled monitors alone executed roughly 85 times per day ($\approx 1,100$ executions in the window, within 1,664 blueprint-workflow runs overall) — and merge-time gating of agent-authored change at exactly this tempo is the phenomenon under study.

6.1 Deployment topology

Structural conformance runs as a **required merge check in four repositories**: the platform monorepo (repository ruleset 18213704 requires the `blueprint-conformance` context), the engine’s own home repository (`blueprint-conformance-local-agents` and `blueprint-self-conformance`), the agent-runtime repository (`blueprint-conformance-odin-agent`, scope-intersected to the agent subtree), and the control-plane repository (`blueprint-conformance`). The second of these realizes self-dogfooding as a design pattern: **the engine gates its own repository** — a change to bce cannot merge if it violates bce’s own blueprint. A fifth repository — the tenant-facing command-center — runs the same conformance gate on every pull request,

but **advisory rather than required** (471 runs as of 2026-08-01; the check is not among that branch’s required contexts). We therefore phrase the deployment throughout as *required merge checks in four repositories, advisory in a fifth*.

Across the five repositories, **33 engineering blueprints** are committed on the default branches (9 / 5 / 9 / 5 plus the served-runtime sidecar / 5, as of 2026-08-01). Among them is one shared blueprint committed byte-identically in all five repositories — the N-party shared-blueprint pattern whose cross-repo byte drift the nightly portfolio sweep polices (§6.5) — though portfolio-level multi-party anti-drift remains future work, not a demonstrated capability (§8). The gated repositories deploy to a multi-tenant fleet of three tenants operated by the author (§6.4).

We report the verification method itself, because it corrected our own documentation. Internal state documents still described several gates as advisory; the claims above were established by reading branch-protection and ruleset state through the live GitHub API, which supersedes the stale doc states. One methodological pitfall is worth recording: enforcement is heterogeneous. One repository enforces through a repository *ruleset*, while another enforces through *classic branch protection* and returns an empty list from the rulesets API — a rulesets-only audit would falsely conclude that no enforcement exists. Auditing required-check claims demands probing both surfaces.¹

Topology addendum (2026-08-10). The required footprint has widened since the freeze, in the advisory-first-then-promote pattern the deployment was designed around: the formerly-advisory command-center gate is now a *required* check on its default branch, bringing the count to **required merge checks in five repositories**; a sixth repository (a standalone intelligence-layer experiment) adopted the gate advisory-first and is now the advisory member. The long-stale 0.6.0 advisory pin was re-pinned to 0.12.1; at 2026-08-10 five consumers pin 0.12.1 and the platform monorepo’s gate has moved to 0.13.0. The toothlessness oracle now runs inside the conformance gates of four of the five required-check repositories (all but the recently-promoted command-center gate), completing the wiring that §4.1 reported as in review at the freeze.

A second engine migration then played out under the same monitoring, and was caught by it mid-flight. On 2026-08-09 the fleet’s declared-intent registry (and the bridge blueprint of §5.3) moved to 0.13.0 ahead of the consumers, and the nightly portfolio sweep immediately went NON-PASS with per-member PIN-DRIFT rows naming the exact gap — extracted pin 0.12.1 against declared intent 0.13.0 — on two consecutive nights. Consumer convergence followed on 2026-08-12: at writing, the platform monorepo, the engine repository, and the agent-runtime repository’s gates install 0.13.0, with the command-center bump queued behind its required checks and the control-plane and advisory bumps in review behind their human-approval gates — pins still move at different speeds, and the monitoring still names each laggard nightly until it converges. En route, 0.13.0’s per-constraint fail-closed refusal did exactly what it exists to do: it exposed one declared-but-unenforced `customPolicy` constraint in the command-center portfolio — green under 0.12.1’s skip-with-note, measured 100→60 under 0.13.0’s refusal — which was retired forward with a recorded rationale in the blueprint itself, its protection re-carried by four route-level tests, rather than skipped or weakened.

¹Engine-pin state is heterogeneous across the fleet, and it moved during the reporting window. As of 2026-08-02, after the pin-lockstep reconcile narrated in §6.5 (Incident 1): the platform monorepo’s five blueprint workflows pin engine 0.12.1; the agent-runtime repository’s four blueprint workflows pin 0.12.0 (since 2026-07-30); the control-plane repository pins 0.12.0 via its lockfile; the engine repository’s gate pins 0.12.0 (since 2026-08-01); the advisory fifth repository still pins 0.6.0 (deliberately unmoved — re-pinning it is a named follow-on). We make no single fleet-wide pin-version claim; the honest statement is that pins move at different speeds, and §6.5 shows the monitoring catching exactly that.

Pin-state addendum (2026-08-31). The pattern has since repeated at the next engine generation, and the monitoring is still the instrument that refuses to call it done. The declared-intent registry now names engine 0.17.0 for all five members, and the platform monorepo’s nine workflow engine literals converged onto it in the week of 2026-08-24; the control-plane repository has not — its `package.json` still pins 0.14.0 (its workflow surface sits behind a human-only approval wall) — and the nightly portfolio sweep has said so every night since: NON-PASS with a per-member PIN-DRIFT row naming the control-plane member on five consecutive nights (2026-08-27 through 2026-08-31), fleet score 100 over the four evaluable members. One further split deserves naming: the command-center’s *runtime* execution path — the in-product code path that runs the engine for tenants, distinct from its CI workflows — still pins 0.12.1 as of 2026-08-31, so a fleet-pin claim scoped to CI workflows is not a claim about the runtime. Pins still move at different speeds; the monitoring still names each laggard nightly until it converges.

6.2 The behavioral gate, green- and red-proven

The behavioral gate `blueprint-behavioral-odin-agent` is a *required* status check on the agent-runtime repository’s main branch — alongside the structural conformance check and an adversarial-verify attestation check, enforced via classic branch protection (verified by live API reads, 2026-07-30/31). On every pull request it executes the served-runtime probe against the built system, then grades the resulting observations with `bce run --observations` against the sidecar served-runtime blueprint of §5 (its 7 behavioral teeth; the subject’s other 10 structural teeth live in the static gate’s blueprint, which is the whole point of the split). All four blueprint workflows on the repository are pinned to the exact engine version 0.12.0, so the graded semantics are reproducible against a fixed evaluator. (The admin-bypass caveat that qualifies every required-check claim in this paper is disclosed in §7.)

A fail-closed gate is only credible if it has been observed to fail, so we report both colors from CI history. The gate is **green-proven** by run **30588771822** on a clean feature branch — engine install, served-runtime probe and blueprint grading all executed and passed — and **red-proven** by run **30584805213** on the planted-regression branch `proof/behavioral-gate-red-fixture-1785121400`, where the probe’s self-guard refused to emit a sidecar that would not honestly pass the reader and failed the run. We name only these two, and we say why the two sibling runs in the same series are *not* gate verdicts: in run 30586345158 the workflow determined the pull request touched no probed seam and skipped the engine install, probe and grading (a *skip*-green, correct path-scoping but not an executed gate), and in run 30584274320 the engine install itself failed against the registry, so the probe and grading never ran. Counting either as gate evidence would be the vacuous pass this paper exists to condemn, so one executed green and one executed red are what we claim. The red runs are the load-bearing evidence: they demonstrate, on the production repository with the production workflow and the pinned engine, that a green verdict from this check is a survived refutation attempt, not an unexercised default. Behavioral CI checks are notoriously flake-prone, so the flake record matters: as of 2026-08-01 the gate has executed **24 times as a required check — 21 green, 3 red** — with zero flakes among the 21 green runs and all three reds accounted for (the two planted-regression runs above and one red on a development branch).

This is a *gate-level* red proof. Two transitions must be kept distinct. A red→fix→green round-trip *was* durably recorded in the window: Incident 1’s monitor and portfolio reds and their post-fix greens are committed rows in the fleet score-series, whose integrity is git history (§6.5). What has not yet been recorded is such a transition on the hash-chained evidence records themselves: the evidence sink that appends chain links has a disclosed buffering defect (§6.5) that has kept the

chains frozen at their 2026-07-23 links. A chain-recorded round-trip remains future validation (§8).

6.3 Standing operation

The gates are standing machinery, and this section reports their operational record. Table 4 gives the per-workflow record.

The aggregate: **1,664 blueprint-workflow runs in the system’s first two weeks, 1,063 of them runs of the six required merge-gate workflows** — the gate sat in the merge path more than a thousand times — and **235 of the platform monorepo’s gate runs fired inside the merge queue**, where the gate re-evaluates against the queue’s combined state (it went red there twice, on 2026-07-27; both are the infrastructure failures of §6.5, Incident 2). We deliberately do not present the ~98% green rate as “the fleet is conformant”: per-PR gates are green largely because authors fix reds before merge; the interesting reds are itemized in §6.5.

Cost, because a millisecond-fast binary checker invites the question: the structural gate’s median wall-clock is **27 s (p90 37 s; n=100)** and the behavioral gate’s median is **20 s (max 56 s; n=21)**, computed from run start/end timestamps (read 2026-08-01). That is what numeric scoring, evidence emission, and served-runtime observation cost at CI time.

Operation has continued past the window at the same tempo: by 2026-08-12 the six repositories’ blueprint workflows total **4,291 runs all-time, 3,333 of them in the trailing fourteen days** (per-workflow GitHub API run totals over fifteen blueprint workflows — part of the growth is roster, not only traffic: five of the fifteen workflows were themselves added within that fortnight, contributing 287 of the runs), and the gate-scoped blueprint portfolio has grown from 33 to **57** gate-discovered blueprints across the six repositories (counting the engine repository’s self-conformance blueprint). The window figures above remain the first-two-weeks record; the teeth audit of §4.1 has not been re-run over the enlarged portfolio, so the 33/33 result stays scoped to its 2026-08-01/02 measurement date. A later re-derivation extends the series: by 2026-08-31 the same per-workflow GitHub API totals count **16,438 runs all-time, 9,661 of them in the trailing fourteen days**, over seventeen blueprint workflows across the six repositories — with the same roster disclosure as before, because part of the growth is again roster rather than only traffic: the enforcement-ledger gate (a required check born ~2026-08-23; the disposition-ledger paragraph below reports what it grades) alone accounts for 3,358 of the all-time runs, and the command-center behavioral gate’s 378 runs all fall inside the trailing window. The 2026-08-12 figures stand as their dated measurement.

Drift is graded at three altitudes: (i) the per-PR required checks above; (ii) a **daily drift monitor** (07:17 UTC) that re-grades the main branches so drift landing between pull requests is detected within a day; and (iii) a **nightly portfolio sweep** (06:47 UTC) that re-runs conformance across the blueprint portfolio, complemented by a weekly anti-shelfware sweep (§6.5).

Post-merge, **evidence sinks are live in three of the five repositories**, at uneven maturity. The platform monorepo’s committed score series carries **63 records over 16 distinct mainline revisions and 4 subsystems (as of 2026-08-01)**, with blueprint versions evolving *inside* the series while under continuous gating (**dark-factory-signal-surface** 0.1.0→0.3.0); the control-plane repository’s committed series is length 1; the agent-runtime repository commits per-cycle evidence sets rather than a top-level series. The mainline series is uniformly green **because the gate is fail-closed** — reds cannot reach main through the normal merge path, so they live in the PR and queue runs (Table 4) and at the monitor and portfolio altitudes (§6.5). A committed coverage-frontier artifact tracks blueprint presence across 9/9 subsystems and carries its own scope disclaimer (“PRESENCE ONLY” — no invented coverage percentages). The series has since kept growing under continuous gating: by 2026-08-31 the platform monorepo’s committed score series carries **1,485 records over 69 mainline revisions and 47 distinct blueprint-at-version**

series — the 2026-08-01 figures stand as the dated start of that curve.

Disposition honesty, one altitude up (2026-08-23 onward). The platform monorepo now also maintains an *enforcement-disposition ledger*: a committed register recording, for every fleet blueprint, whether a *required* check actually grades it (**enforced**), a required check grades only a subset of its constraints (**partial** — a demotion carrying a hard expiry, never a resting state), a non-required check merely reports on it (**advisory**), or it is authored but not yet governed (**draft**); a validator recomputes scope resolution from the checkout on every run and refuses any disposition the tree cannot back, and that validator is itself wired as a required merge check. As of 2026-08-31 the ledger carries **75 entries: 43 enforced, 1 partial, 23 advisory, 8 draft**. This is the toothlessness ratchet one altitude up: a blueprint can be authored-and-green while no required check grades it, and the ledger makes that authored-versus-enforced gap a first-class, gated fact rather than something a reader must re-derive by hand.

Human-legible projections with their own truth-gates (2026-08-29). The evidence layer has additionally gained a human-visuals family — an atlas builder, a per-blueprint prospectus generator, and a cross-linking index that project the machine verdicts into stakeholder-legible pages sharing one glossary and hero grammar. The honesty discipline extends to the generated prose: author-time truth-gates (run as `node --test suites`; deliberately not yet wired into CI, and we state that rather than imply otherwise) parse the *rendered* output and fail on any numeral that lacks a machine source, negative-controlled twice — a smuggled “saves 40 hours” benefit claim and a doctored “99%” template were each caught — with a seeded-red KPI gate and a fail-closed link check (35/35 hyperlinks resolved over the full 34-prospectus estate run). The atlas generator is itself blueprint-governed, with a toothed mutation corpus in which fabricated-verdict, hardcoded-perfect-score and skip-flag mutants must each redden.

6.4 Fleet context

All production data in this paper comes from a single estate: a **multi-tenant fleet of three tenants, operated by the author**, on which the gated repositories deploy. There is no external adoption, no pilot customer, and no third-party deployment behind any number reported here. The deployment is real and load-bearing (the gates block the author’s own merges daily), but it is self-referential; §7 treats this as the leading threat to validity.

6.5 Operational events and lessons

The window produced three narratable real incidents, and the week after it two more (Incidents 4 and 5, dated 2026-08-08/10). We report all five at forensic depth before the standing lessons, because depth-per-event is what a short window can offer.

Incident 1 — the organic version-skew red, and its verified fix (2026-08-01/02; Figure 3). The daily drift monitor’s scheduled run on 2026-08-01 (run 30690870648) went red with verdict DRIFT: a blueprint authored against $\geq 0.9.0$ constraint vocabulary (`forbiddenPattern`, `minEngineVersion`) had landed on the main branch, and the monitor’s own engine leg was still pinned at 0.6.0 — the older engine’s strict schema refused to parse the newer vocabulary, scored the blueprint 0, and exited 1, loudly. This is §3.8’s thesis demonstrated by an unplanned production event: an older engine confronted with newer semantics failed **closed and loud**, never

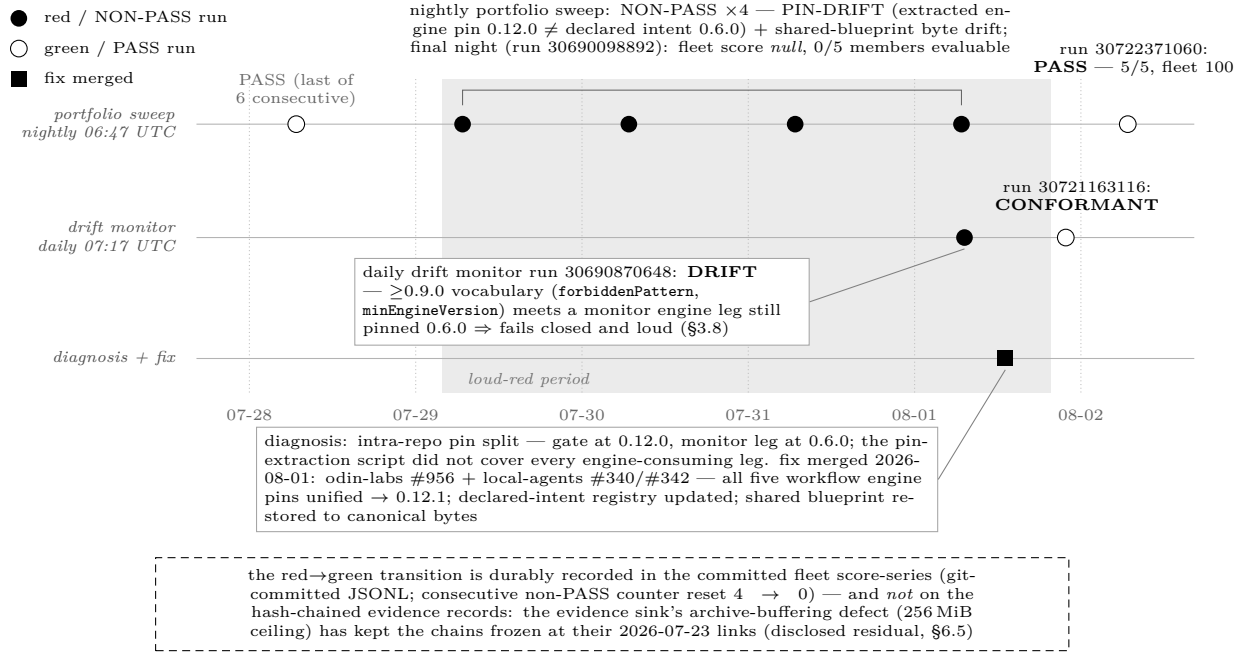


Figure 3: Incident 1 (§6.5): the organic version-skew red and its verified fix, 2026-07-29 $\rightarrow$ 2026-08-02 (all run numbers are CI workflow-run ids). The nightly portfolio sweep withheld green for four consecutive nights — PIN-DRIFT (extracted engine pin 0.12.0 $\neq$ declared intent 0.6.0) plus shared-blueprint byte drift, with an honest fleet score of *null* (0/5 members evaluable) on the final night — and on 2026-08-01 the daily drift monitor went red organically: a blueprint carrying $\geq 0.9.0$ constraint vocabulary met a monitor engine leg still pinned at 0.6.0 and failed closed and loud (§3.8). The pin-lockstep reconcile (odin-labs #956 with local-agents #340/#342) unified all five of the monorepo’s workflow engine pins at 0.12.1, and the red $\rightarrow$ green transition was verified on real post-merge runs (drift monitor 30690870648 $\rightarrow$ 30721163116 CONFORMANT; portfolio sweep 30690098892 $\rightarrow$ 30722371060 PASS, fleet score 100) and durably recorded in the committed fleet score-series — not on the hash-chained evidence records, whose sink defect is a disclosed residual.

silently green. The complication: the intra-repo skew (gate at 0.12.0, monitor at 0.6.0) had survived the portfolio pin-extraction script, because the script’s coverage did not extend to every engine-consuming workflow leg. In the same week the nightly portfolio sweep had been refusing to call the fleet green for four consecutive nights (2026-07-29 → 08-01): the committed verdict series shows six consecutive PASS nights, then the fleet’s 0.12.x re-pin week, in which the sweep reported PIN-DRIFT (“extracted engine pin 0.12.0 != declared intent 0.6.0”) and shared-blueprint byte drift, and on the final night an honest fleet score of *null* with 0 of 5 members evaluable — withheld beats invented. These are fleet-conformance-*process* defects (pin drift, shared-blueprint byte drift, a missing artifact), not architecture violations in the graded code; catching them is what the portfolio altitude exists for.

The fix was a pin-lockstep reconcile merged 2026-08-01 (odin-labs PR #956 with local-agents PRs #340/#342): all five of the monorepo’s workflow engine literals unified — to 0.12.1, after examining that 0.12.0’s full-repo AST scan crashes on the estate’s bash-subsystem blueprints and that the 0.12.1 patch removes the crash — the declared-intent registry caught up, and the shared blueprint restored to canonical bytes. The red→green transition was then **verified on real workflow runs**, whose triggers we state precisely rather than generically: the two red runs fired on the nightly **schedule**, and the two green runs were operator-triggered re-runs (**workflow_dispatch**) — not post-merge events. Drift monitor run 30690870648 (**schedule**, DRIFT, failure) → run 30721163116 (**workflow_dispatch**, CONFORMANT, success); portfolio sweep run 30690098892 (**schedule**, NON-PASS, 0/5 members evaluable, fleet score null) → run 30722371060 (**workflow_dispatch**, PASS, 5/5 members evaluable, fleet score 100). One precision on the red sweep: run 30690098892’s *run-level* conclusion is **cancelled** (one member’s gate job was cancelled), while its **portfolio verdict** job concluded **failure** and wrote the NON-PASS row. We report both labels, because the NON-PASS is a job-level fact and the run-level label alone would misstate it. The transition is durably recorded in the **committed fleet score-series**: the red row and the recovery row are both git-committed JSONL on the main branch, with the heartbeat’s consecutive-non-pass counter reset 4→0.

One disclosure remains: the transition is *not* recorded on the hash-chained evidence records, because the evidence sink that appends chain links has a pre-existing defect — it buffers a **git archive** of the tree under a 256 MiB ceiling that the repository’s archive now exceeds — which has kept the chains frozen at their 2026-07-23 links. The sink defect is a disclosed residual, and the chain-recorded round-trip remains unclaimed (§6.2, §8).

Incident 2 — the admin bypass, exercised once, measured (2026-07-27). The required structural gate failed twice inside the merge queue on the same pull request (runs 30276262830 and 30274924006) — job-level failures with no failed step, i.e. an infrastructure failure of the gate’s own execution, not a red verdict — and the operator merged the PR directly. The PR was a security fix. §7’s admin-bypass disclosure therefore carries data rather than hypothesis: **the bypass was used once in the window, against an infrastructure failure of the gate itself, not to override a red verdict.** (The failed-step logs had expired by read time; we cite the two runs as red merge-queue events without asserting a deeper cause.)

Incident 3 — the gate resists its own upgrade (2026-07-31). The control-plane repository’s pull request re-pinning its engine to 0.12.0 first went red (run 30663429950) before landing in a consistent state — though not at the gate itself: the run failed one step earlier, installing the pinned engine from a frozen lockfile, so **Run bce gate** never executed and is recorded **skipped**. That is the honest shape of this incident, and it is still the point: upgrade friction on the pin-lockstep path

is real and fails closed and loud — including, and especially, when the change under test is the gate’s own engine.

Incident 4 — green-while-broken: a structurally toothed, behaviorally vacuous blueprint (2026-08-08). The operator-steering bridge blueprint at v0.4.0 — ten static constraints, all machine-checkable, 10/10 toothed by the oracle of §4.1 — scored 100/*pass* while the subsystem’s inbound leg was operationally dead. Nothing in the static gate was wrong: every constraint it could grade held. What the blueprint lacked was any constraint that could observe the *served* behavior, so the verdict was the §1 flip one level up — teeth against the dependency graph, vacuity against the running system. The remediation (§5.3) added four input-conditioned **behavioralInvariant** constraints, each red-before/green-after proven against the real producers; the blueprint can no longer be green while that loop is broken, and the proof arc includes the gate catching a deliberately seeded output regression (score 20, constant-function oracle) before it could ship. The transferable lesson sharpens §5.2: the toothlessness oracle certifies that *some* realistic mutation reddens the blueprint — it does not certify that the blueprint’s constraint set spans the subsystem’s behavioral surface. Span is an authoring judgment; the behavioral-invariant machinery is what makes that judgment enforceable once made.

Incident 5 — the fixture-vs-live observation boundary (2026-08-09/10). The steward re-grade of §5.3 graded the bridge blueprint green (score 100, freshly regenerated observations) at 05:45 UTC on 2026-08-09 while the subsystem’s live delivery transport was degraded: per the runtime audit log, the last read-back-verified threaded delivery landed at 03:46 UTC, and from 04:16 UTC every delivery fell back to a webhook path carrying no post identifier and no read-back — a worktree-local credential file had vanished (bracketed 03:46–04:16 UTC by the log), and the fallback branch emitted no audit signal of its own. The measured outage ran ~19 h (restore effective 23:00 UTC, read-back-verified deliveries continuous thereafter). The steward was not wrong: its delivery-invariant observation producer exercises the real pipeline with the outbound HTTP hop stubbed through the consumer’s own documented test hook, so a live-credential loss sits outside what those observations can witness. We report this as a *coverage boundary*, not a grading defect — the same distinction §7 draws for the recall corpus’s recorded-probe scope. The fix-forward record: the silent fallback was made loud (an audit verdict naming the unresolvable credential, red-before/green-after proven in the wrapper’s suite), and the credential path was restored; closing the boundary itself — an end-to-end live-transport observation arm graded alongside the stubbed one — is future work (§8).

The converse discipline: declare-restraint (2026-08-11). A third production subject applied the inverse of the bridge remediation. A developer-workstation blueprint (five structural hygiene constraints, gated and green) gained a read-only served-runtime probe that measured seven behavioral observations against the live container — and two of the three candidate invariants those observations would feed measured as *would-violate* (both credential-shaped findings, forwarded to the operator). The invariants were then deliberately *not* declared: the required check’s CI context cannot reach that runtime, so a declared invariant would fail closed on every pull request regardless of the runtime’s actual state — a permanent red that grades the CI topology, not the system. The two honest shapes this deployment has converged on are exactly the bridge pattern (declare, and grade where observations exist — accepting the recorded CI-side score-0 as the ledger’s truthful statement that CI cannot see behavior) and the workstation pattern (measure first, declare only where a grading surface exists). Declaring behavioral intent a gate can never observe is a third

shape, and it is vacuity wearing a seatbelt.

Continuation evidence (2026-08-30/31). The discipline the five incidents teach has kept firing since the freeze, and three fresh instances are worth recording because each is a checker-honesty class this paper names, caught on the estate’s own governance surfaces rather than on graded product code. First, a muted-pager defect (platform-monorepo PR #2735, merged 2026-08-31): an operator-alert vault’s writer and reader resolved *different* directories, so a default-path signing validated in a directory the paging path never consults — and the surrounding test suite structurally could not see it, because every test case exported the very environment variable whose effect it asserted: a test that supplies what it asserts, the fixture-blindness class of Incident 5 on a test suite instead of an observation producer. The fix landed with a seeded-red proof — seven assertions fail against the pre-fix writer, including “writer signed but reader did not arm,” the incident shape verbatim. Second, a vacuous-success red on the estate’s own lint (PR #2738, merged 2026-08-31): a footer validator whose selection produced an empty population had been reporting success over zero subjects; the fix makes the empty population refuse — the same *a-green-verdict-must-mean-something-was-proven* invariant this paper applies to blueprints, enforced on the toolchain that checks the estate, and it turned the main branch honestly red on landing. Third, blueprint authoring again worked as a detection instrument: grounding a messaging-lane blueprint clause surfaced a live fallback code path that a prior repository-wide sweep of the same defect class had missed (PR #2662, merged 2026-08-30), and the class was then frozen out by a four-constraint blueprint landed with a two-sided red/green proof and its enforcement-ledger row (PR #2669) — the incident-response authoring mode of the paragraph below, exercised again.

Red taxonomy. We call a red *organic* when the gate executed and emitted a failing verdict on a change that was not authored to make it fail: not a planted drift-proof branch, and not an infrastructure failure of the gate’s own execution (in which the gate step never runs). The structural required gates of Table 4 carry nine reds in the window, which decompose exactly: four planted (the platform monorepo’s three drift-proof branches of 2026-07-19/20, plus the control-plane’s [RED proof, do not merge] branch, run 29683676540); four infrastructure (the platform monorepo’s two merge-queue failures of Incident 2, and the control-plane’s runs 29683658084 and 30663429950, both of which failed at dependency install with Run bce gate recorded skipped); and one organic. In 13 days and 1,039 structural required-gate executions (as of 2026-08-01; the remaining 24 of the 1,063 required-gate runs are the behavioral gate’s, §6.2), **the per-PR gates produced exactly one organic red verdict** — a version-skew parse red on the agent-runtime repository (run 30577591956, 2026-07-30), the same failure class as Incident 1 and seven days earlier, at the PR altitude — **and zero organic architecture-violation reds**; the organic architecture reds appeared at the drift-monitor and portfolio altitudes instead (Incident 1). Part of the explanation is expected and worth stating: the same agents that write the code read the blueprints, so the per-PR gate acts as a deterrent whose effect is unmeasurable at N=1 organization — a named limitation (§7). The early portfolio-sweep failures (ten, clustered 2026-07-19→23) are bootstrap-era shakeout, not caught drift; the sweep’s meaningful signal starts with its committed verdict series (15 rows through 2026-08-01 — nightly from 2026-07-23, with four same-evening re-runs on 08-01 — plus the 2026-08-02 post-fix verification row).

Teeth by default. The motivating incident of §1 was also an operational one: the flagship subject graded 100/pass under a vacuous blueprint and 0/fail with 75 authentication-boundary violations under a toothed blueprint over the same ~60-file route surface, flipped through the product surface

itself. The lesson is that a conformance verdict inherits the honesty of its blueprint. The measured consequence across the whole portfolio is §4.1’s teeth audit: 33/33 fleet blueprints toothed, one genuine authored vacuity found at the constraint level, four unwitnessable constraints quarantined (2026-08-01/02, engine 0.12.0) — evidence that keeping a vacuity oracle in the authoring loop drives measured fleet vacuity to approximately zero, which §4.1’s oracle-version caveat scopes as a lower bound under the superseded oracle. The oracle’s CI wiring is reported in §4.1.

The required-check wedge. A required check must never be path-filtered at the workflow level: a pull request touching none of the filtered paths never dispatches the workflow, the required context never reports, and the PR wedges un-mergeable. Our gate workflows therefore trigger unconditionally and perform scope intersection *inside* the tool, where “nothing in scope” can still report an explicit verdict.

Pin-lockstep requires coverage of every leg. Upgrading the engine in some gate workflows but not others produces silent cross-repo skew in what the gates can enforce. A portfolio pin-extraction script blocks partial engine upgrades — and the window’s own telemetry shows both the value and the limit of that mechanism: the script exists, yet the fleet still skewed on legs the script did not cover (the monorepo’s monitor workflow and the declared-intent registry), and it was the three-altitude monitoring, not the lockstep script, that caught the skew — four consecutive loud nights of PIN-DRIFT and a withheld (null) fleet score until the reconcile landed (Incident 1). The lesson as we now state it: lockstep requires coverage of every engine-consuming leg, and the monitors are what tell you when it fails.

Version-skew honesty is a fleet necessity, not a nicety. Because pins inevitably move at different speeds, the engine’s current semantics — unknown constraint types surface as explicit advisories, never silent passes; `minEngineVersion` yields a clear error — are what prevent an older engine from quietly disabling exactly the newest teeth. Incident 1 is this lesson exercised in production.

Incident response is an authoring mode. Blueprint authoring turned out to be an incident-response mode, not only a design-time one: a staging-ingress outage on 2026-07-31 was codified — within hours of root-cause, in the same repository as the incident record — as an enforceable six-constraint blueprint (all `critical`) forbidding the direct provider-SDK reach that caused it, merged the same day (odin-labs PR #907). Fix-forward, captured as intent.

Anti-shelfware. A weekly sweep re-asserts the zero-blueprints floor on a cadence: it evaluates every authored blueprint and refuses to report green if the discovered count reaches zero across both extractor legs, so the gate cannot be silently retired by deleting the portfolio. The sweep additionally fails if any discovered blueprint is non-conformant, which catches a component vanishing when no pull request touches its scope. We are precise about what this does *not* cover, because the shipped predicate is a comparison against zero: a *gradually* shrinking discovery set that never reaches zero is not detected. Catching partial portfolio decay requires a declared-count baseline to compare against, which we have not built; we name it as a limitation rather than claim it.

7 Threats to Validity

Self-referential evidence. Every production observation in §6 comes from the author’s own estate and fleet. This is the accepted shape of a first-party experience report, and we say it plainly: the paper demonstrates that the mechanisms operate under real merge pressure on a real deployment, not that they transfer to other organizations. No external adoption is implied anywhere in this paper.

Corpus provenance and scope. The measured-recall result (recall 1.000, 25/25, all defect-specific; false-positive rate 0.000 over 35 reports) rests on N=25 seeded defects across 23 distinct seeded fixtures and 12 clean fixtures, covering all eight enforcing constraint types including three defects in the motivating **C-AUTH-02** class (as of 2026-08-02, local-agents PR #345, merge f8d6a900). Four limits remain. First, every defect was seeded by the paper’s single author: the corpus measures the checker against the defect classes its own builder anticipated, and defects harvested from external deployments remain future work (§8). Second, the two **behavioralInvariant** entries measure the engine’s observation-grading arm over *synthetic* observation documents — committed fixtures hand-authored to the **behaviorObservation** shape, not recordings of a running system — merged through the real fail-closed **--observations** path; neither the live probe (stimulus → served runtime → hash recording) nor the authenticity of the observation document is inside the fixture corpus’s measurement scope — and Incident 5 (§6.5) shows the practical bite of exactly this scope class in production: an observation producer that stubs the outward hop cannot witness a live-credential outage. Third, a perfect score over this corpus (25 at the freeze; 34 at the 2026-08-10 re-measurement) is evidence of checker competence on these defect classes, not a general detection-rate estimate; the rigor story is the protocol — defect-specific matching with an executable expected-fire matrix, the cried-wolf rule, on-disk corpus integrity enforced in CI, byte-determinism across runs, and pinned widen-only thresholds — not the sample size. Fourth, the clean fixture set is scoped to the engine’s *declared support envelope*: one known false-RED case outside that envelope — a route whose guard arrives through an imported wrapper (**withTenantGuard(handler)**), which the extractor cannot credit and would flag despite the route being genuinely guarded — is documented as a coverage boundary and deliberately excluded from the clean set, so the measured false-positive rate is a statement about code inside the declared envelope, not about arbitrary conformant code.

Fixtures, not live routes. The corpus defects were seeded into fixture corpora *derived from* production extraction surfaces and measured by the real extractor and evaluator end-to-end; they are realistic drifted variants of real surfaces, not live production routes or live traffic. We use this phrasing deliberately and consistently.

Forbidden-shape vocabulary: green means no drift found, not required structure present. Eight constraint types are enforcing, but only two — **requiredComponent** and **requiredDependency** — can fail on an *absence*, and the deployed portfolio barely uses them. Across the five repositories’ default branches on 2026-08-14, the 79 committed ***.engineering-blueprint.json** files carry 466 constraints, of which 432 (92.7%) are forbidden-shape; 26 (5.6%) are statically-gradeable presence-shape, spread over 20 blueprints, so 59 of the 79 carry none at all. A further 8 are **behavioralInvariant**, which the static gate skips by design. The consequence is a coverage boundary rather than a grading defect: a forbidden-shape constraint is evaluated over the components the extractor observed, so a subject that has been *emptied* rather than drifted presents nothing to forbid and scores 100. The minimum-file floor of

§3.4 bounds this at exactly one file — a scan of zero files fails closed; a scan of one near-empty file does not. We can demonstrate the boundary on our own shipped example: replacing the body of the quickstart’s conformant plugin with a comment leaves `bce run` at score 100/*pass* with zero violations and `bce teeth` at `toothed -- 1/1`, both byte-identical to the genuinely conformant tree (`--ct-repo clean --no-pin --extractor ast`). Reading a green structural verdict as “the required architecture is present” is therefore unsound unless the blueprint carries presence-shape constraints, and most of ours do not. Re-run whole on 2026-08-31 over the same five default branches, the census has grown to 91 committed files (14 of which are the engine package’s committed fixture and demo blueprints; the remainder are deployed-portfolio artifacts) carrying 501 constraints: 459 (91.6%) forbidden-shape, 26 (5.2%) statically-gradeable presence-shape spread over 22 blueprints — so 69 of the 91 still carry none — and 16 `behavioralInvariant`. Excluding the fixture and demo files sharpens rather than softens the point: 448 of the deployed portfolio’s 471 constraints (95.1%) are forbidden-shape. Growth has not narrowed the boundary this paragraph reports. Raising presence-shape coverage, and adding a deletion-mutation arm to the toothlessness oracle, are future work (§8).

The observation channel is trusted. `bce` grades `behavioralInvariant` constraints from an observation document supplied on the command line. The engine validates that document’s shape and rejects it fail-closed if malformed, but it performs no authentication of its provenance: nothing in the engine distinguishes a document a probe recorded from a running system from one a developer wrote by hand. Integrity of that channel is therefore a property of the surrounding wiring rather than of the engine — in the deployment reported here the probe executes inside the same CI job that grades its output and self-guards against emitting a sidecar that would not honestly pass the reader, and the steward re-grade regenerates every observation set from the running system immediately before grading — and we state it as a limitation rather than claim a guarantee we have not built. Cryptographic attestation of observation provenance is future work (§8).

Admin bypass. The gates are required checks, but on the behavioral-gate repository’s main branch `enforce_admins=false` and `required_approving_review_count=0`: an administrator merge can bypass the required contexts. The fail-closed guarantee is therefore a property of the normal merge path, not an absolute one; we disclose rather than hide this configuration. The disclosure carries a measured instance rather than a hypothesis: the bypass was exercised once in the reporting window — 2026-07-27, on a security-fix PR, after the gate failed twice in the merge queue on an infrastructure failure of its own execution rather than a red verdict (§6.5, Incident 2).

One organic PR-gate red. In the reporting window the per-PR required gates produced exactly one organic red verdict — a version-skew parse red (run 30577591956), not an architecture violation — while the organic architecture reds appeared at the drift-monitor and portfolio altitudes instead (§6.5). The likely mechanism — the same agents that write the code read the blueprints — makes the gate’s deterrence effect unmeasurable at a single organization. We own that near-zero rather than hide it: it is a real limitation of a single-organization deployment, not evidence that the gate works. Measuring deterrence requires more than one deployment.

Single author; AI assistance. This paper has a single author, and both the system and the manuscript were developed with substantial AI assistance, disclosed per venue policy. The evidence discipline — every claim traced to a verified artifact, locked phrasings for anything unverified — is in part a mitigation for exactly this authorship shape.

Hedge scope. The “to our knowledge” conjunction claim rests on absence of evidence after a hard prior-art search dated 2026-07-31, re-run on 2026-08-28 with the conclusion unchanged — the movement found (SonarQube’s successor capability going generally available; active archgate releases; adjacent community and position work) sharpens the mainstream baseline of §2 without combining the conjunction; five of the nine closest works are themselves 2026-dated, and a final pass precedes posting.

Independence of concurrent work. SGE [12] was submitted to arXiv on 25 June 2026. Our first engine commit is dated 18 July 2026 (86dff27b, “Blueprint Engineering walking skeleton”), our first registry publish 19 July 2026 (v0.2.0), the end-to-end measured-recall proof 21 July 2026, and the behavioral required-check promotion 30 July 2026 — a first-commit-to-measured-production-gate arc of under two weeks, all subsequent to SGE’s submission date. We became aware of SGE on 31 July 2026, through our own pre-publication novelty search; no design element of bce derives from it, and the full dated commit history ships with the released repository. We therefore claim independence of development, not temporal precedence: the priority this paper asserts is realization and measurement — a shipped, production-gated, self-validated system — which SGE, as a design proposal without a released tool or evaluation, does not contest.

8 Limitations and Future Work

Multi-repo portfolio conformance. The nightly sweep aggregates per-repository results, but portfolio-level anti-drift — multiple parties gated against shared blueprints across repository boundaries — has been exercised only on a single shared tree and is future work, not a demonstrated capability.

Excluded machinery. Adjacent evidence-pack automation is deliberately excluded from this paper’s claims: a scheduled compliance-evidence-pack job exists in configuration but is not installed (no trigger; blocked on operator review), and we make no claim in either direction about the downstream customer-facing trust-reporting surface. Nothing reported here depends on either.

Growing the corpus. With per-constraint-type coverage complete (all eight enforcing types, §4.3), the natural growth path for the seeded-defect corpus is defects harvested from external pilot deployments rather than author-seeded classes — which would both raise N and reduce the self-referential character of the validation — together with a live-probe measurement arm for `behavioralInvariant` that would close the recorded-observations scope caveat of §7. Similarly, a witnessed red → fix → green round-trip reflected in a lengthening *hash-chained* evidence record remains to be demonstrated; it awaits the evidence-sink buffering fix disclosed in §6.5.

Teeth in CI. At the freeze, the oracle was wired into two of the four required-check repositories; as of 2026-08-10 it runs inside the conformance gates of **four of the five** (now-five) required-check repositories — all but the recently-promoted command-center gate (§6.1, addendum). The teeth audit itself has not been re-run over the since-enlarged portfolio (57 gate-discovered blueprints at 2026-08-12); the 33/33 result stays dated.

Attestation predicates. Expressing bce’s hash-chained evidence records as in-toto/SLSA-style attestation predicates, so conformance verdicts ride existing supply-chain provenance rails, is a direction we identify — not machinery we have built.

Composition conformance. Extending the same authored-intent-versus-observed-evidence grading shape from repository structure to the runtime composition of agent harnesses — which components, capabilities, and providers may compose for a given unit of work, under whose rules — is a direction we identify, not machinery we have built.

Business-value altitude. The blueprint schema has since gained an optional business-value layer mapping constraints to the outcomes they protect (merged upstream 2026-08-29); no published engine version carries it yet, and nothing reported in this paper depends on it.

Generalizability. The extraction surface is no longer TypeScript-only: the open-core lineage ships a pluggable extraction-profile registry with a Python import-surface extractor whose undetected constructs are each pinned by an asserting test (§4.3), with verdict stability proven against the legacy path. The primary extractors remain AST-based (ts-morph) and Node-centric, the Python arm covers import-surface constraints only, and transfer to other stacks and organizations remains unmeasured.

The fixture-vs-live observation boundary. Incident 5 names a standing limitation of recorded-observation grading: producers that stub the outward hop grade the pipeline, not the live credential/transport state, so a green behavioral verdict coexisted with a degraded live transport for ~19 hours. An end-to-end live-transport observation arm — graded alongside the stubbed arm so that credential-class outages redden the same blueprint — is identified, not built.

The longitudinal agenda. What a two-week window cannot buy, a follow-up report will have to: constraint-set evolution over months, blueprint maintenance cost, alert fatigue under standing monitors, and defect classes harvested from an external pilot. This is the planned longitudinal v2 of this experience report.

9 Conclusion

The same code, graded twice, produced a perfect score under a blueprint that could not fail and a score of zero — with 75 authentication-coverage violations — under one that could. That flip is the paper in miniature. We have reported the design and first production deployment of bce, a deterministic blueprint-conformance engine whose loop operates as required merge checks under real merge pressure: 1,063 required-gate runs in its first two weeks and 16,438 blueprint-workflow runs by 2026-08-31; a checker validated by measurement (recall 1.00 at a false-positive rate of 0.00 over a seeded-defect corpus grown append-only from 25 to 34 defects spanning all eight enforcing constraint types, with a zero-survivor mutation hunt); a portfolio-wide toothlessness audit (33/33 blueprints toothed at its measurement date); an organic version-skew red that failed closed and loud before being fixed and verified green; and a structurally toothed blueprint caught certifying a behaviorally dead subsystem, remediated with input-conditioned behavioral invariants that were then observed catching a seeded regression. We have not shown transfer beyond one estate; deterrence is unmeasurable at $N=1$; the chain-recorded round-trip remains future validation. A conformance verdict is only as honest as the machinery that could have produced a red one; we have reported what it costs, and what it catches, to run that machinery for real.

10 Availability

The engine’s production lineage is a self-contained package (18 published versions from 0.2.0 to 0.17.1 — fifteen minor releases plus three patches, 0.2.1, 0.12.1 and 0.17.1, the 0.16.x minor never published; 0.17.1 latest as of 2026-08-31) on a private, access-restricted registry; every production run of §6 executed from that lineage. Its runtime dependencies are `zod` and `ts-morph` only, with zero internal imports; the build ships the fixture corpora and a gate-workflow template. **The open-core release of the verifier is public:** <https://github.com/blueprint-conformance/bce>, under Apache-2.0, with the engine published on the public npm registry as `bce-engine` (0.1.2 at the time of writing; repository visibility, anonymous installability, and the npm version re-verified 2026-09-01) — the open surface being the schema, extraction (including the Python import-surface profile), `evaluate()`/scoring, gate verb, teeth, evidence emission, and the version-skew semantics of §3.8. The corpus-v3 measurements of §4.3 were taken in that open-core lineage at a pinned revision, and the open-core repository carries the genericized, namespace-migrated protocol reproduction of the seeded-defect corpus with its end-to-end proof tests, publicly re-executable; its `corpus/CORPUS-MAP.md` states the claim and the non-claim precisely — the protocol reproduces, and the files are by construction not byte-identical to the internal originals. The open-core release publishes the blueprint format under a vendor-neutral, domainless schema namespace (`apiVersion: blueprint-conformance/v1alpha1`, with schema documents served from the project organization’s GitHub Pages), while every run recorded in this paper used the original internal namespace (`odin.ai/v1alpha1`). **A paper-artifacts package — the complete seeded-defect corpus and fixture trees, the corpus manifest, sample hash-chained evidence records with a zero-dependency chain verifier, and the CI transcripts of the red and green gate runs cited in §6 — is publicly available at <https://github.com/odin-labs-ai/bce-paper-artifacts>, public since 2026-09-01 with every removal or transformation relative to the internal originals declared in its `EXCLUSIONS.md`, and will be archived under a Zenodo version DOI minted at the posting-day deposit.** Chain integrity re-derives with node alone (`node verify/verify-chain.mjs evidence/born-public`); the frozen N=25 measurement of §4 was taken on the internal lineage, whose package remains available to reviewers on request.

The residual tradeoff of this shape: the frozen N=25 measurement (the internal engine’s 2026-08-02 freeze) is publicly *inspectable* — corpus, manifest, and recorded measurement transcripts ship in the artifacts package — while public *re-execution* runs the open-core lineage’s genericized protocol reproduction (the lineage in which the N=34 re-measurement itself was taken; §4.3) rather than the byte-identical internal fixtures; the artifacts package maintains the byte-frozen originals so genericized and frozen rows can be compared defect-by-defect.

The headline validation result re-derives as follows; the first command is publicly runnable from an open-core checkout, the second from the artifacts package, and the third requires the internal engine package (on request):

```
# protocol reproduction (public): real extractor + evaluator over the
# genericized corpus, gate verdict at the pinned thresholds
#           [checkout of github.com/blueprint-conformance/bce]
npm ci && npx vitest run tests/recall-e2e-proof.test.ts \
    tests/corpus.test.ts tests/corpus-manifest.test.ts
# evidence-chain integrity: zero-dependency verifier over shipped
# records                                     [artifacts package]
node verify/verify-chain.mjs evidence/born-public
```

```
# frozen N=25 engine-level measurement [internal package, on request]
pnpm --filter @odinlabs-ai/engineering-blueprint test recall-e2e-proof
```

Data availability. The CI transcripts of the cited workflow runs (green: 30586345158, 30588771822; red: 30584805213, 30584274320) and the dated branch-protection and ruleset API reads — including the `enforce_admins=false` disclosure of §7 — ship in the artifacts package, as do sample hash-chained evidence records, committed score-series excerpts, and the seeded-defect corpus with its manifest (N=25, matching §4). The primary artifact behind the §1 motivating example (the 75-violation C-AUTH-02 proof record) was committed to the author’s repository on 2026-08-01 and ships in the artifacts package in sanitized form. Raw fleet series beyond the shipped samples remain available on request. The corpus and its end-to-end proof tests additionally ship inside the engine package; the genericized reproduction is public in the open-core repository.

References

- [1] act101. act101: AST-aware AI agents for code refactoring, porting, and architecture conformance. <https://act101.ai/>, 2026. Accessed 2026-07-31.
- [2] Amazon Web Services. Kiro: The AI IDE for spec-driven development. <https://kiro.dev/>, 2026. Accessed 2026-08-14.
- [3] ArchGate. ArchGate CLI: Enforce architecture decision records as executable rules for humans and AI agents. <https://github.com/archgate/cli>, 2026. Apache-2.0. Documentation: <https://cli.archgate.dev/>. Accessed 2026-07-31.
- [4] Ilan Beer, Shoham Ben-David, Cindy Eisner, and Yoav Rodeh. Efficient detection of vacuity in ACTL formulas. In *Computer Aided Verification, 9th International Conference, CAV ’97*, volume 1254 of *Lecture Notes in Computer Science*, pages 279–290. Springer, 1997. doi: 10.1007/3-540-63166-6_28.
- [5] Ilan Beer, Shoham Ben-David, Cindy Eisner, and Yoav Rodeh. Efficient detection of vacuity in temporal model checking. *Formal Methods in System Design*, 18(2):141–163, 2001. doi: 10.1023/A:1008779610539.
- [6] Clinton Cao, Simon Schneider, Nicolás E. Díaz Ferreyra, Sicco Verwer, Annibale Panichella, and Riccardo Scandariato. CATMA: Conformance analysis tool for microservice applications. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings (ICSE Companion ’24)*, 2024. doi: 10.1145/3639478.3640022. arXiv:2401.09838.
- [7] Matteo Casserini, Alessandro Facchini, and Andrea Ferrario. Beyond the “diff”: Addressing agentic entropy in agentic software development, 2026. arXiv:2604.16323. Position paper, HCXAI Workshop at CHI 2026; v2 2026-04-21.
- [8] Humberto Cervantes, Rick Kazman, and Yuanfang Cai. Improving LLM-assisted code generation through the use of architectural documents and implementation plans. In *Proceedings of the Designing 2026 Workshop (co-located with ICSE 2026)*, 2026. URL <https://conf.researchr.org/details/icse-2026/designing-2026-papers/2/>

- Improving-LLM-assisted-code-generation-through-the-use-of-architectural-documents-and. Presented 13 April 2026.
- [9] Sabry E. Farrag. The productivity-reliability paradox: Specification-driven governance for AI-augmented software development, 2026. URL <https://arxiv.org/abs/2605.01160>. arXiv:2605.01160.
 - [10] Neal Ford, Rebecca Parsons, and Patrick Kua. *Building Evolutionary Architectures: Support Constant Change*. O’Reilly Media, 2017. ISBN 978-1491986363.
 - [11] GitHub. Spec Kit: Toolkit for spec-driven development. <https://github.com/github/spec-kit>, 2026. MIT. Accessed 2026-08-14.
 - [12] Hartwig Grabowski. The spec growth engine: Spec-anchored, code-coupled, drift-enforced architecture for AI-assisted software development, 2026. URL <https://arxiv.org/abs/2606.27045>. arXiv:2606.27045.
 - [13] Orna Kupferman and Moshe Y. Vardi. Vacuity detection in temporal model checking. *International Journal on Software Tools for Technology Transfer*, 4(2):224–233, 2003. doi: 10.1007/s100090100062.
 - [14] Jingwen Liu, Wuxia Jin, Junhui Zhou, Qiong Feng, Ming Fan, Haijun Wang, and Ting Liu. 3erefactor: Effective, efficient and executable refactoring recommendation for software architectural consistency. *IEEE Transactions on Software Engineering*, 50(10):2633–2655, 2024. doi: 10.1109/TSE.2024.3449564.
 - [15] Gail C. Murphy, David Notkin, and Kevin J. Sullivan. Software reflexion models: Bridging the gap between source and high-level models. In *Proceedings of the 3rd ACM SIGSOFT Symposium on Foundations of Software Engineering (SIGSOFT FSE ’95)*, pages 18–28. ACM, 1995. doi: 10.1145/222124.222136.
 - [16] Gail C. Murphy, David Notkin, and Kevin J. Sullivan. Software reflexion models: Bridging the gap between design and implementation. *IEEE Transactions on Software Engineering*, 27(4):364–380, 2001. doi: 10.1109/32.917525.
 - [17] ogil109. Blueprint index — living architecture map (Spec Kit community-extension submission). <https://github.com/github/spec-kit/issues/3628>, 2026. v0.2.0; submitted 2026-07-22; tiered CI gate blocking on critical map-versus-code contradictions; the author’s own scope statement: “The gate does detection, not conformance”. Accessed 2026-08-28.
 - [18] OpenSSF. SLSA: Supply-chain levels for software artifacts, version 1.2. <https://slsa.dev/spec/v1.2/>, 2025. OpenSSF / Linux Foundation. v1.0 retired; v1.2 current as of 2026-07-31.
 - [19] Deepak Babu Piskala. Spec-driven development: From code to contract in the age of AI coding assistants, 2026. URL <https://arxiv.org/abs/2602.00180>. arXiv:2602.00180. Submitted to AIWare 2026.
 - [20] Oleg Solozobov. DEMM-Bench: A cross-regime benchmark for agent-runtime governance-evidence sufficiency, 2026. URL <https://arxiv.org/abs/2606.20634>. arXiv:2606.20634.
 - [21] SonarSource. Design & architecture: Architecture as code (sonarqube server 2025.2). <https://docs.sonarsource.com/sonarqube-server/2025.2/design-and-architecture/overview>, 2025. Introduced in SonarQube Server 2025 Release 2. Accessed 2026-07-31.

- [22] SonarSource. Introducing SonarQube Server 2026 Release 4. <https://www.sonarsource.com/blog/introducing-sonarqube-server-2026-4/>, 2026. Architecture management (intended-vs-actual, deviations flagged in the quality gate) generally available in Server, July 22, 2026; first announced for SonarQube Cloud (beta) December 16, 2025 (<https://www.sonarsource.com/blog/introducing-architecture-in-sonarqube/>). Accessed 2026-08-28.
- [23] Tessl. Tessl: Spec-driven development and the AI native registry. <https://tessl.io/>, 2026. Accessed 2026-08-14.
- [24] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium (USENIX Security '19)*, pages 1393–1410. USENIX Association, 2019. URL <https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias>.
- [25] vFunction. vfunction: The architectural observability and governance platform. <https://vfunction.com/>, 2024. Architecture governance announced 2024. Accessed 2026-07-31.
- [26] Charles L. Wang, Trisha Singhal, Ameya Kelkar, and Jason Tuo. MI9: An integrated runtime governance framework for agentic AI, 2025. URL <https://arxiv.org/abs/2508.03858>. arXiv:2508.03858.
- [27] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE 2026)*, 2026. URL <https://arxiv.org/abs/2503.18666>. arXiv:2503.18666.

Table 1: Differentiation against the closest prior systems.

Axis	bce (this paper)	SGE [12]	archgate [3]	SonarQube arch. [21, 22]	act101 [1]	vFunction [25]
Authored-intent artifact	✓ operator-authored, PR-reviewed, schema-strict blueprint (§3.1)	✓ spec graph — design only	partial — ADRs + <code>.rules.ts</code>	✓ YAML/JSON definition	×	×
Numeric deterministic score	✓ 100 — Σ severity, byte-deterministic (§3.3)	×	×	×	×	×
Fail-closed required-check gate	✓ required checks, 4 repos; per-constraint fail-closed (malformed, under-min-version, under-min-files, zero-enforcing all FAIL); no CLI skip flag (§3.4, §6.1)	designed; no tool	✓ CI check blocking merges	partial	✓ but refactor/port scope only	×
Hash-chained re-derivable evidence	✓ content-addressed, chained (§3.5)	×	×	×	partial — signed receipts	×
Checker self-validation (measured recall)	✓ seeded-defect corpus, N=34 (append-only from 25), all 8 enforcing types; 1.00/0.00 + zero-survivor mutation hunt (§4)	×	×	none published	none published	none published
Toothlessness/vacuity gate	✓ bce teeth (§4.1)	×	×	×	×	×
Served-runtime behavioral observations	✓ inside a required check, green+red-proven (§5, §6.2)	×	×	×	partial — test-reachability	✓ but no gating vs intent
Shipped tool vs design	✓ shipped (14 registry versions as of 2026-08-10)	×	✓ OSS CLI	✓ shipped; successor GA in Server 2026.4	✓ commercial	✓ commercial
Evaluation present	✓ recall measurement + production data	×	×	×	×	×
Production deployment	✓ author’s own fleet — self-referential, disclosed (§6)	×	unknown	commercial installs	vendor	commercial

Table 2: Constraint taxonomy. Evidence class per `constraintEvidenceClass()`: *staticAst* (facts from the source extractor) or *behaviorObservation* (facts from the served-runtime probe).

Constraint type	What it grades	Evidence class	Status
<code>forbiddenDependency</code>	A violation for each observed import edge from $\rightarrow$ to; <i>from</i> may be <code>*</code> (“any component”)	<code>staticAst</code>	enforcing
<code>requiredDependency</code>	Every observed component of the target type must carry a satisfying outgoing edge (e.g. a route handler with no <i>guards</i> edge is a violation)	<code>staticAst</code>	enforcing
<code>requiredComponent</code>	At least one observed component of the named type must exist; zero $\rightarrow$ violation	<code>staticAst</code>	enforcing
<code>forbiddenPath</code>	Any extracted component whose path matches the forbidden path	<code>staticAst</code>	enforcing
<code>forbiddenFile</code>	Filename glob against the raw scanned-file set — catches surfaces that mint no components (added 0.8.0)	<code>staticAst</code>	enforcing
<code>forbiddenEgress</code>	A call-egress edge to a provider HTTP host; blocklist (<code>to</code>) or allowlist (<code>governedHosts</code>) mode — catches raw <code>fetch()</code> calls that carry no import edge	<code>staticAst</code>	enforcing
<code>forbiddenPattern</code>	Per-line regex over raw scanned-file content (ReDoS-safe; the regex must compile at authoring time) — catches mocked data inside otherwise-legitimate files (added 0.9.0)	<code>staticAst</code>	enforcing
<code>behavioralInvariant</code>	≥ 2 distinct stimuli must produce distinct observed outputs and satisfy the scenario’s property oracle (the constant-function/mock detector)	<code>behavior-Observation</code>	enforcing (run mode; the static gate skips it with an explicit advisory)
<code>requiredEvidence</code>	—	—	schema-reserved; explicit skip
<code>minimumMetric</code>	—	—	schema-reserved; explicit skip
<code>customPolicy</code>	—	—	schema-reserved; explicit skip

Table 3: Teeth verdicts over the production blueprint portfolio (33 CI-gated blueprints, five repositories; measured 2026-08-01/02 at pinned engine 0.12.0 and pinned default-branch heads).

Level	TOOTHED	TRIVIALY_GREEN	INDETERMINATE	Total
Blueprint-level verdict	33 (100%)	0 (0%)	0 (0%)	33
Constraint-witness level	212 (97.7%)	1 (0.46%)	4 (1.84%)	217

Table 4: Blueprint-workflow operations across the five repositories (all runs since each workflow’s first run; read 2026-08-01).

Repository	Workflow (role)	First run	Runs	Green	Red	Canc.
platform monorepo	conformance gate (required)	07-19	554	549	5	0
platform monorepo	drift monitor (daily cron)	07-22	13	11	2	0
platform monorepo	evidence sink (post-merge)	07-19	24	20	4	0
platform monorepo	portfolio sweep (nightly cron)	07-19	27	13	10	4
platform monorepo	fleet-loop advisory	07-31	12	11	1	0
engine repository	conformance gate (required)	07-21	100	100	0	0
engine repository	self-conformance gate (required)	07-20	93	92	0	1
control-plane repository	conformance gate (required)	07-19	152	149	3	0
control-plane repository	evidence sink (post-merge)	07-20	34	34	0	0
agent-runtime repository	conformance gate (required)	07-19	140	139	1	0
agent-runtime repository	behavioral gate (required)	07-30	24	21	3	0
agent-runtime repository	evidence sink (post-merge)	07-21	17	17	0	0
agent-runtime repository	fleet-refresh advisory	07-31	3	3	0	0
command-center repository	conformance gate (advisory)	07-21	471	466	4	1
Total			1,664	1,625	33	6